arXiv:2611.10165v1 [cs.AI] 6 Nov 2026

AUTOEMATIC: RUNTIME-ADAPTIVE MIXED-PRECISION LLM INFERENCE VIA AUTOMATIC EXPONENT–MANTISSA REFINEMENT

Akshat Ramachandran[†,1] **Reena Elangovan**[2] **Rangharajan Venkatesan**[2] **Charbel Sakr**[2] **Tijmen Blankevoort**[2]
Brucek Khailany[2] **Tushar Krishna**[1]

ABSTRACT

Existing LLM post-training quantization techniques determine precision before inference begins, yet the precision a model requires varies dynamically across prompts, layers, and inference phases. We present AutoEMatic, a full-stack co-design that adapts the precision of weights, activations, and the KV cache at runtime. A lightweight policy selects each layer's precision at every prefill and decode step from two runtime signals: the entropy of the next-token distribution and the magnitude of each layer's update to its hidden states. This runtime adaptation is made practical by AutoFP, a multi-scale floating-point format that derives 2-, 4-, and 6-bit precisions from a single NVFP4 base format. We realize AutoEMatic on two targets: GPU kernels that execute AutoFP on NVFP4 Tensor Cores, and an accelerator whose vector MAC datapath executes all AutoFP formats natively. Across reasoning and agentic benchmarks, AutoEMatic improves accuracy by up to 21.7% over NVFP4 and matches group-scaled FP6 accuracy on most benchmarks, using 4.9 bits per weight versus 4.5 for NVFP4. On GB300 GPUs, AutoEMatic retains approximately 90–98% of NVFP4 decode throughput across the evaluated configurations. Furthermore, the AutoEMatic accelerator reduces latency by 1.6–2.3× and energy by 7–23% compared to an accelerator with NVFP4-only datapath at iso-accuracy.

1 INTRODUCTION

AI, driven by LLMs, is reaching an inflection point as the focus shifts from training to efficient inference at scale. Rising request volumes and long-horizon agentic workloads (Kariyappa & Suh, 2026) strain inference infrastructure in compute, memory bandwidth, and energy. Post-training quantization (PTQ) techniques improve inference efficiency by lowering the precision of weights, activations, or KV cache. Existing PTQ techniques (Lu et al., 2026; NVIDIA ModelOpt, 2025), however, employ static precision assignments, determined before the start of inference rather than adapting to the model's evolving numerical demands during generation. These numerical demands vary dynamically with the prompt, between prefill and decode, and from one decoding step to the next. Therefore, a static assignment over-provisions precision where it is unnecessary and vice-versa. This motivates **runtime precision adaptation**: dynamically adjusting precision across prompts, layers and inference phases to match a model's numerical demands.

Recent works have taken initial steps toward this goal. FlexQuant (Liu et al., 2025) switches layer-wise weight precision during decoding but stores separate quantized weight copies. Multi-scale quantization (Nair et al., 2025) avoids these copies by nesting several precisions within one integer bit layout whose width equals the highest supported precision. From this nested layout, lower precisions are obtained by truncating the least significant bits. DP-LLM (Kwon et al., 2026) extends this, switching each layer at decode time between two offline-calibrated precisions according to an input-dependent estimate of quantization error.

These approaches leave three challenges. First, their policies are not true runtime adaptations as they rely on offline calibration. This limits their ability to accurately track evolving precision demand during inference (Figure 1 (amber region)), yielding a suboptimal accuracy–precision trade-

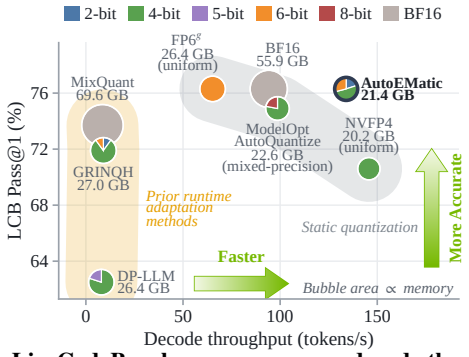


Figure 1. **LiveCodeBench accuracy versus decode throughput for Qwen3.8-27B** (batch size 1, one GB300 GPU).

[†]Work done during an internship at NVIDIA Research. [1]Georgia Institute of Technology [2]NVIDIA. Correspondence to: Akshat Ramachandran <akshat.r@gatech.edu>.

Under Review, Copyright 2026 by the author(s).

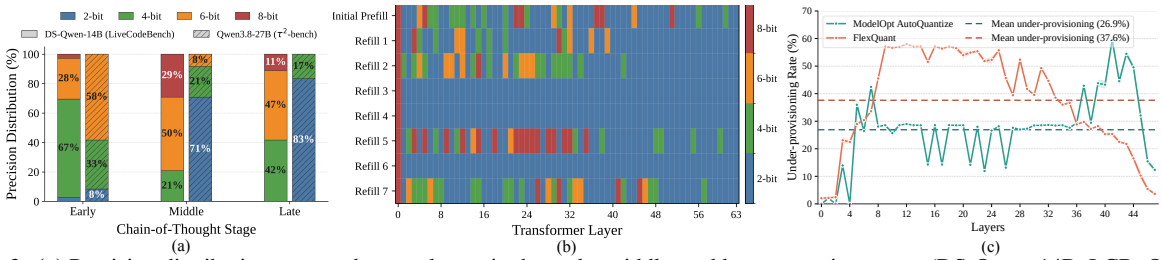


Figure 2. **(a)** Precision distribution averaged across layers in the early, middle, and late generation stages (DS-Qwen-14B, LCB; Qwen3.8-27B, τ^2-bench). **(b)** Layer-wise precision assignments for initial prefill and additional refills (Qwen3.8-27B, τ^2-bench). **(c)** Rate of underprovisioning for ModelOpt AutoQuantize and FlexQuant (DS-Qwen-14B, LCB).

off. Second, existing multi-scale formats use integer representations that suffer substantial accuracy degradation when lower precisions (sub-6-bit) are extracted from the nested representations, especially for activations and the KV cache (§5). Their mechanism does not directly extend to group-scaled floating-point formats such as NVFP4, which is used for low-precision inference on NVIDIA Tensor Cores. Moreover, extracting precisions from their nested representations incur irregular memory accesses and dequantization overheads (Park et al., 2026), that sacrifice inference performance. Finally, they do not address batched inference, where one layer must serve requests of different precisions.

Contributions. To address these limitations, we introduce **AutoEMatic**. At its core is a format-agnostic runtime policy that selects each layer's precision across prefill and decode; AutoFP makes the policy practical by deriving all precisions from one NVFP4 base, and GPU kernels and an accelerator demonstrate AutoEMatic on existing and custom hardware. These four components form a full-stack co-design:

- *Runtime precision adaptation policy* (§3.1): We identify two model-internal signals, a per-token and a per-layer measure of precision demand, and derive from their behavior a set of rules that assign layer-wise precisions.
- *Quantization format* (§3.2): We introduce *AutoFP*, the first multi-scale floating-point (FP) format, which composes 2-, 4-, and 6-bit precisions from a shared NVFP4 base: the foundation from which other formats are composed. AutoFP6, the 6-bit E2M3 precision is obtained through mantissa refinement: by adding mantissa bits selectively to high quantization error base elements. The refinement metadata is stored separately from the base and applied on demand based on the runtime policy. Conversely, AutoFP2, the 2-bit precision is a quinary format, obtained by remapping the exponent bits of the base precision.
- *System implementation* (§4): We integrate AutoEMatic into vLLM with kernels that execute AutoFP using NVFP4 Tensor Cores and efficiently serve concurrent requests.
- *Hardware accelerator* (§6): We design an accelerator with an AutoFP-native mixed-precision datapath, which can process two AutoFP2 weights, one NVFP4 weight, or one AutoFP6 weight against a single streamed input activation.

To the best of our knowledge, AutoEMatic (Figure 1) is the first framework to jointly adapt weight, activation, and KV-cache precision at runtime, across prefill and decode, using an FP-based multi-scale format. Across reasoning and agentic benchmarks on models including Qwen3.8-27B, Gemma4-26B, and MiniMax-M2.7, AutoEMatic **improves accuracy by up to 21.7% over NVFP4** and **matches FP6^g** (the group-scaled 6-bit counterpart to NVFP4) on most benchmarks, using **4.9 bits per weight** versus 4.5 for NVFP4. AutoEMatic sustains this accuracy advantage while retaining **90–98% of NVFP4 throughput**. On custom hardware, AutoEMatic reduces latency by 1.6–2.3× and energy by 7–23% compared to an NVFP4-only datapath at iso-accuracy.

2 Motivating Analyses

2.1 Precision Demand Changes During Inference

To understand how the precision across layers of a model change throughout inference, we set up an oracle that determines for each layer at every prefill and decode step, the lowest precision (WxAx-KVx) such that the KL-divergence between the quantized and full-precision model's next-token distributions are within 10^{-3} nats, maintaining iso-generation. Following a full-precision generation, each layer is quantized sequentially, accounting for cumulative quantization effects. The oracle chooses between the following candidates: NVFP4, group-scaled 2-bit (E1M0) and 6-bit (E2M3) counterparts to NVFP4, and standard FP8 (E4M3).

Decode Variations. We apply this analysis to responses from DS-Qwen-14B on LiveCodeBench (LCB) and Qwen3.8-27B on τ^2-bench, with up to 10K tokens per turn. Figure 2(a) aggregates the oracle precisions assignments across layers within the early, middle, and late thirds of each response. The distribution varies across decoding stages and models, reflecting underlying layer-wise precision variations from one decode step to the next (§C.1, Appx.).

Prefill Variations. In agentic workloads, returns from tool calls extend the context and trigger additional prefills (Lu et al., 2026), termed *refills*. Figure 2(b) shows Qwen3.8-27B's layer-wise oracle precision assignments over the ini-

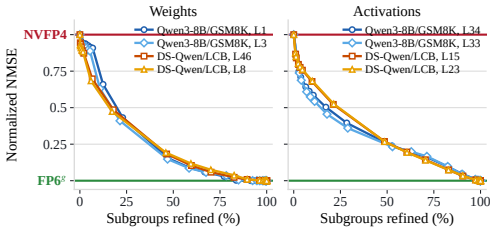


Figure 3. **Refining a subset of elements achieves NMSE comparable to 6-bit**. Y-axis is NMSE, such that NVFP4 is 1 and FP6^g is 0, vs. fraction of subgroups refined to 6-bit.

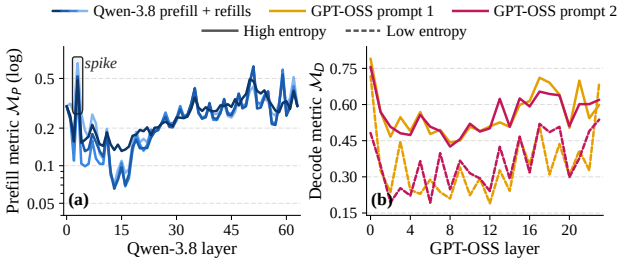


Figure 4. **Layer-wise behavior of $\mathcal{M}_P$ and $\mathcal{M}_D$.** (a) Prefill $\mathcal{M}_P$ of Qwen3.8-27B on a τ^2-bench task. (b) Decode $\mathcal{M}_D$ of GPT-OSS-20B on RULER-16K, at high- and low-$\mathcal{G}_\mathcal{E}$ steps.

tial prefill and refills. Demand changes sharply between prefill stages: refill 3 uses 2-bit for nearly all layers, while others demonstrate a broader precision distribution (§C).

Therefore, precision demand depends jointly on the model, prompt, layer, and inference stage, and can change with any of these factors. The challenge of runtime-precision adaptation is to identify these dynamic variations without access to the full-precision reference used by the oracle.

2.2 Limitation of Static Mixed-Precision Assignment

We further ask whether a static assignment calibrated in-distribution (calibrated and evaluated on the same dataset) can closely track the oracle's precision assignments. We examine two schemes: ModelOpt AutoQuantize (NVIDIA ModelOpt, 2025), which fixes per-layer precisions using calibrated sensitivities, and FlexQuant (Liu et al., 2025), which follows a static schedule of lowering precision during inference. Both use 100 LiveCodeBench calibration samples under a 6-bit budget, matching the oracle's achieved average precision, and are evaluated on the remaining 780 samples. Figure 2(c) reports underprovisioning: the fraction of decode steps where the oracle requires higher precision for a layer than the static policy assigns. AutoQuantize under-provisions 26.9% on average and FlexQuant 37.6%. In-distribution calibration tailors to the aggregate precision demands of the calibration samples. However, the resulting fixed assignments or schedules do not respond to fine-grained variations across prompts and decode steps, leading to under-provisioning even on held-out samples from the same dataset, which directly impacts accuracy (§5).

2.3 Non-uniform precision requirement within groups

Raising the precision (refining) of a tensor, for example from 4 to 6 bits, is conventionally applied uniformly (Ramachandran et al., 2024). We instead ask whether the error of a group-scaled format such as NVFP4 is concentrated in a few elements of each group, so that refining those elements alone suffices. We partition each 16-element group into four 4-element subgroups ($SG = 4$; §I (Appx.)) and sweep NMSE thresholds from 0.3 to 0.001, and map the elements of every subgroup above the threshold to 6-bit (E2M3). For Qwen3-8B and DS-Qwen-14B (Figure 3), refining $\sim 50\%$ of weight subgroups achieves $\sim 90\%$ of the NMSE reduction obtained by moving from NVFP4 to FP6^g uniformly. Activations exhibit a similar trend. AutoFP leverages this to reduce metadata overhead when constructing higher precisions from NVFP4.

3 AutoEMatic Quantization

3.1 Runtime Precision Adaptation Policy

Precision-Adaptation Signals. AutoEMatic determines precision at runtime from two complementary model-internal signals. The first is a global token-generation uncertainty signal $\mathcal{G}_\mathcal{E}$, measured as the normalized Shannon entropy of the top-K tokens' output logits (§D.1). When the model is uncertain about the next token (high entropy), small quantization induced perturbations in the output logits can change which token it generates (Lee et al., 2026b;a). Such steps warrant higher precision and vice-versa. However, a global signal alone cannot determine layer-wise precision variations. The second signal is based on the layer structure of LLMs. LLM layers have a residual connection that carries its input directly to its output, with the attention and MLP blocks adding their contributions along the way (Zhao et al., 2026). A layer whose output hidden states $\mathbf{H}_\ell(t)$ are close in representation to its input $\mathbf{H}_{\ell-1}(t)$ contributes little beyond copying its input to the output; we term it a low-activity layer, and its weights, activations, and KV cache can tolerate lower precision. This local layer-update activity $\mathcal{L}_\mathcal{A}$ is measured at every step t, for every layer l as,

$$\mathcal{L}_\mathcal{A}{}^\ell(t) = \frac{\|\mathbf{H}_\ell(t) - \mathbf{H}_{\ell-1}(t)\|_2}{\|\mathbf{H}_{\ell-1}(t)\|_2 + \varepsilon}, \tag{1}$$

where ε ensures numerical stability. The two signals combine into one precision-demand metric for prefill or decode (P/D), $\mathcal{M}^\ell_{P/D}(t) = \lambda_{P/D}\,\mathcal{L}_\mathcal{A}{}^\ell(t) + (1-\lambda_{P/D})\,\mathcal{G}_\mathcal{E}(t)$, with $\lambda_{P/D} \in [0,1]$ set separately for each phase[1]. Rather than calibrated thresholds, AutoEMatic tracks the running mean μ and standard deviation σ of $\mathcal{M}_{P/D}$ and establishes a set of rules, that vary across inference stages: sustained increases (decreases) of the metric over a fixed number of preceding layers relative to μ and σ raise (lower) the precision of the

[1] $\mathcal{M}^\ell_{P/D}$ is a joint notation, $\mathcal{M}^\ell_P$ in prefill and $\mathcal{M}^\ell_D$ in decode.

Table 1. **Runtime precision adaptation policy.**

Policy	Prefill	Decode	
		High entropy	Low entropy
Metric	$\mathcal{M}_P = \mathcal{L}_\mathcal{A}$	$\mathcal{M}_D = 0.7\,\mathcal{L}_\mathcal{A} + 0.3\mathcal{G}_\mathcal{E}$	
Initial precision	6-bit	4-bit	4-bit
Rules activate	After spike	Layer 0	
Raise precision	n_p layers $> \mu$	β layers $> \mu$	α layers $> \mu$
Lower precision	n_p layers $< \mu$	α layers $< \mu$	β layers $< \mu$

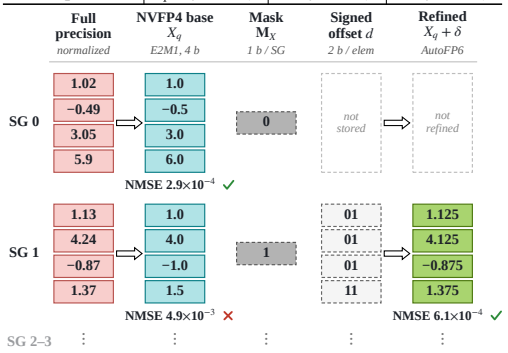


Figure 5. **Mantissa refinement of NVFP4 to AutoFP6.**

next layer. Intuitively, these rules observe sustained trends in the metric as evidence that precision should change, rather than reacting to isolated fluctuations. Each layer executes at WxAx-KVx with $x \in \mathcal{P} = \{2, 4, 6\}$ chosen by the rule. 2-bit activations and KV cache degrade accuracy substantially, therefore, the 2-bit level applies to weights only (W2A4-KV4). $\mathcal{P}$ denotes nominal precision levels; the policy is agnostic to the choice of levels or quantization format.

Prefill policy. Prefill precedes token generation, so no token entropy exists and we set $\lambda_p = 1$ ($\mathcal{M}_P = \mathcal{L}_\mathcal{A}$). We observe that, across models and datasets, $\mathcal{M}_P$ shows a single large spike[2] at an early layer, at a position largely invariant across prompts and refills (Figure 4(a)). Consistent with early-exit work (Shen & Zou, 2026), the spike marks the layer by which the model has integrated the prompt context, so the layers up to and including it are the most sensitive and must execute at $\max \mathcal{P} = 6$ bits. Because the spike position differs across models, AutoEMatic detects it online as $\mathcal{M}_P^\ell > \mu + 3\sigma$. The subsequent layers follow the rule, n_p layers below (above) μ lower (raise) the precision of the next layer by one level in $\mathcal{P}$, saturating at its bounds.

Decode policy. During decode, we set $\lambda_D = 0.7$ incorporating generation entropy into the metric (Figure 4(b)). Since token entropy varies little between consecutive steps (Ramachandran et al., 2026) and the $\mathcal{G}_\mathcal{E}$ of the current step is known only after its token is generated, $\mathcal{M}_D$ reuses $\mathcal{G}_\mathcal{E}$ from the preceding step. Every decode step starts layer 0 at 4 bits and applies the rules in Table 1. Since layer tolerance to quantization varies with generation entropy, we separate decoding into *high-entropy* ($\mathcal{G}_\mathcal{E} \geq \tau$) and *low-entropy* ($\mathcal{G}_\mathcal{E} < \tau$) steps by a threshold τ. At high-entropy steps, the rule requires α consecutive low-activity layers to lower precision, but only β consecutive high-activity layers to raise it, with $\alpha > \beta$. That is, AutoEMatic lowers precision more conservatively. At low-entropy, thresholds are reversed.

[2]Characterized by a sudden rise in the metric value.

3.2 AutoFP Quantization Format

Storing one quantized copy per level in $\mathcal{P}$ would cost 13.5 bits per weight. AutoFP instead stores one representation, the NVFP4 base with its group scales and additional metadata, and derives the 2- and 6-bit levels from it on demand.

AutoFP6. Each additional mantissa bit adds local precision around every NVFP4 value. However, a floating-point value cannot be refined by 'appending' mantissa bits. Since NVFP4 rounds to nearest, the full-precision value may lie below the E2M1 value, whereas appended mantissa bits can only select values above it, and not the nearest E2M3 value in the neighborhood that minimizes quantization error (e.g., 1.4 is stored as 1.5 in E2M1; appended bits can then reach only 1.5–1.875, never 1.375).

Therefore, instead of storing the additional metadata as mantissa bits, AutoFP encodes them as *additive signed offsets*. Because the E2M1 grid is a subset of the E2M3 grid, every E2M1 value X_q is also an E2M3 value. Refining X_q only requires storing the direction and number of E2M3 grid steps to move from it. Since E2M3 adds three additional values around every X_q, a 2-bit two's-complement signed offset $d \in \{-2, -1, 0, +1\}$ offers sufficient resolution. The step size around X_q with exponent e is 2^{e-3}, and the refinement term is $\delta = d \cdot 2^{e-3}$ (the mantissa equivalent value of the signed offset) and the refined value is thus, $X_q + \delta$. The refinement is thus additive. Since the step 2^{e-3} is dependent on the exponent of X_q, decoding d exactly would require a per-element exponent computation (§7). AutoFP therefore decodes d with a fixed step 2^{-3}, $\delta = d \cdot 2^{-3} \in \{-\frac{1}{4}, -\frac{1}{8}, 0, +\frac{1}{8}\}$. The fixed step is E2M3-like since it reproduces the E2M3 grid for values < 2 and applies a step finer than E2M3 for larger values.

To reduce the metadata overhead of storing signed offsets, AutoFP6 refines only the subgroups with the largest NMSE reduction (§2.3). Each 16-element group carries a 4-bit refinement mask $\mathbf{M}_X$ with one bit per 4-element subgroup, set to 1 for a refined subgroup and 0 otherwise, and the signed offsets are stored only for the subgroups with mask bit of 1 (Figure 5). The NVFP4 base is left intact, the signed offsets are stored separately and applied on demand for AutoFP6 layers. Since the refinement is additive, every refined operand then splits as $\widehat{\mathbf{X}} = \mathbf{X}_q + \mathbf{M}_X \odot \mathbf{X}_r$ for $\mathbf{X} \in \{\mathbf{W}, \mathbf{A}, \mathbf{K}, \mathbf{V}\}$, where $\mathbf{X}_r = \delta$. By linearity, a refined GEMM is, $\widehat{\mathbf{W}}\widehat{\mathbf{A}} = \mathbf{W}_q\mathbf{A}_q + (\mathbf{M}_W \odot \mathbf{W}_r)\mathbf{A}_q + \mathbf{W}_q(\mathbf{M}_A \odot \mathbf{A}_r) + (\mathbf{M}_W \odot \mathbf{W}_r)(\mathbf{M}_A \odot \mathbf{A}_r)$.

Two choices make this hardware-friendly. First, AutoEMatic imposes N:4 structured sparse (Ramachandran et al., 2025b) refinement within each group, such that N_W weight

and $N_A = 4 - N_W$ activation subgroups are refined per group. Second, it keeps $\mathbf{M}_W$ and $\mathbf{M}_A$ disjoint. $\mathbf{M}_W$ selects N_W subgroups for refinement (along K-dimension) and is shared across the rows of $\mathbf{W}$, and $\mathbf{M}_A$, shared across the columns of $\mathbf{A}$. This ensures disjointness holds for every row–column pair in the GEMM. Disjointness eliminates the second-order product of refinements, which contributes negligibly to accuracy (§7), reducing the number of additional operations required for a refined layer. $\mathbf{M}_A$ is selected at runtime and $\mathbf{M}_W$ is selected offline under a refinement budget ρ, the fraction of weight subgroups refined, either by NMSE or by a refinement-aware GPTQ calibration that assigns refinement for subgroups with the largest Hessian-weighted error (§E).

AutoFP2. Truncating the stored NVFP4 to 2 bits yields an E1M0 format representing $\pm1, \pm2$, which cannot represent zero and causes significant accuracy degradation. AutoFP2 instead is a quinary format, $Q \in \{-2, -1, 0, 1, 2\}$, derived from the NVFP4 code by retaining the sign bit s and remapping the two exponent bits e_1e_0 to a 2-bit magnitude in $\{0, 1, 2\}$ on demand via $\{e_1 \cdot e_0,\ e_1 \oplus e_0\}$. Nominally, representing AutoFP2 requires 3-bits. However, we show in §6, the accelerator packs two AutoFP2 digits into 5 bits, 2.5 bits per digit. AutoFP2 reuses the NVFP4 scales which normalize to $[-6, 6]$; since Q reaches only ±2, the scale is multiplied by a constant c (§5). As AutoFP2 is derived from NVFP4 on demand, it saves no memory bandwidth, and its main benefit is in prefill throughput and energy (§6).

4 System Implementation

We implement kernels that executes AutoFP6 natively on NVFP4 Tensor Cores from a single packed model: E2M1 base values with E4M3 group scales $s_{W/A}$, signed offsets d stored as one byte per refined subgroup, and refinement masks $\mathbf{M}_{W/A}$ stored as 4 bits per group along the K-dimension (§D.3, Appx.). Since Tensor Cores do not support sub-4-bit formats, AutoFP2 layers are emulated as NVFP4, which preserves the accuracy of the precision decision but not the performance of AutoFP2 execution.

4.1 Refinement as an Extended-K NVFP4 Operation

A GEMM accumulates products along its reduction dimension K, so a sum of GEMMs with the same output shape equals one GEMM over operands concatenated along K (see Appx.). AutoEMatic executes the refined GEMM of §3.2 in this manner. Since $\mathbf{M}_W$ and $\mathbf{M}_A$ are disjoint, the two refinement terms can be merged into a single GEMM of up to length-K as, $\mathbf{W}_{qr}\mathbf{A}_{qr}$ (Figure 6). Each of these operands are constructed as follows: for subgroups that refine weights, $\mathbf{W}_{qr}$ holds the refinement term δ_W and $\mathbf{A}_{qr}$ holds the activation base values A_q in the corresponding position, such that $\mathbf{W}_{qr}\mathbf{A}_{qr}$ realizes $(\mathbf{M}_W \odot \mathbf{W}_r)\mathbf{A}_q$. Similarly, for subgroups with activation-refinement, $\mathbf{W}_{qr}$ holds

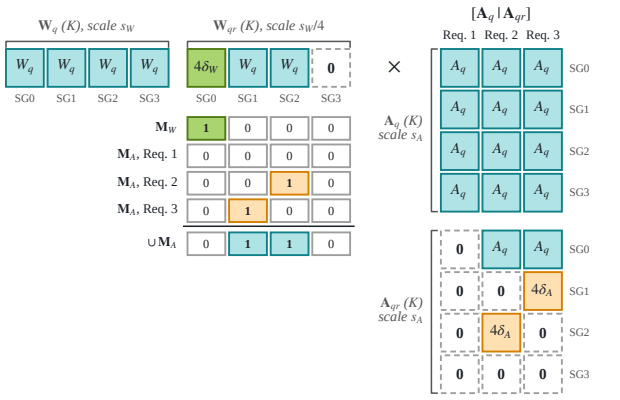


Figure 6. **Extended-K GEMM for one 16-element group and three requests (batch size of 3).**

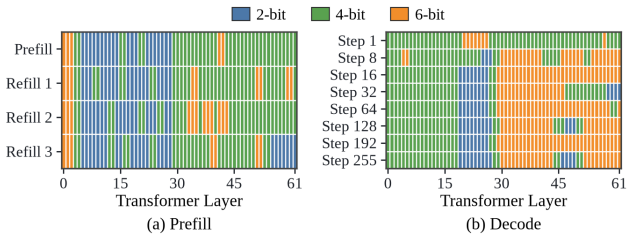
the weight base values W_q and $\mathbf{A}_{qr}$ holds δ_A, realizing $\mathbf{W}_q(\mathbf{M}_A \odot \mathbf{A}_r)$. The reduction dimension is up to length-K because it depends on the number of subgroups refined. A refined layer thus executes as a single NVFP4 GEMM of reduction length up to $2K$ as, $[\mathbf{W}_q \mid \mathbf{W}_{qr}] \cdot [\mathbf{A}_q \mid \mathbf{A}_{qr}]$, which we term the *extended-K GEMM*. The GEMMs use Blackwell's `tcgen05.mma` instruction, which consumes only E2M1 values with E4M3 group scales. The refinement δ is not representable in E2M1 as is, whereas its scaled version $4\delta \in \{-1, -0.5, 0, +0.5\}$ is. The refined subgroups of $\mathbf{W}_{qr}$ and $\mathbf{A}_{qr}$ therefore hold 4δ, and the additional factor of four is absorbed into the scale of $\mathbf{W}_{qr}$, which is set to $s_W/4$. Each refined subgroup then contributes $(s_W/4 \cdot 4\delta_W)(s_A A_q) = s_W s_A\, \delta_W A_q$ or $(s_W/4 \cdot W_q)(s_A \cdot 4\delta_A) = s_W s_A\, W_q \delta_A$. The kernel fuses the construction of both extended operands with the MMA. Attention follows the same construction and runs on Flash-Infer's FP8 attention kernel. As in ThinKV (Ramachandran et al., 2026), AutoEMatic uses two page types of equal byte size: a base page holds T unrefined tokens with only the base values, and a refinement page holds $T/2$ refined tokens with base values and signed offsets in NVFP4. This enables one paged allocator to serve both types.

4.2 Serving Mixed-Precision Batches

Requests in a batch may select different precisions for the same layer. The weight operands do not vary with that choice: $\mathbf{W}_q$ is shared by every request, and $\mathbf{W}_r$ with its mask $\mathbf{M}_W$ is fixed offline. Only the activation masks $\mathbf{M}_A$ and their signed offsets d differ across requests. Rather than running a separate GEMM for each precision decision, AutoEMatic forms the *union of masks* $\cup\mathbf{M}_A$, the per-group OR of the activation masks of all requests in the batch, and serves the whole batch with one extended-K GEMM against a single $\mathbf{W}_{qr}$ built from it. As shown in Figure 6, $\mathbf{W}_{qr}$ holds $4\delta_W$ in its weight-refined subgroups, W_q in every subgroup of $\cup\mathbf{M}_A$, and zero elsewhere. Each request then constructs its own $\mathbf{A}_{qr}$: A_q in the weight-refined subgroups,

Table 2. **Accuracy comparison of AutoEMatic and baselines across reasoning and agentic workloads.** Execution precision appears in parentheses next to accuracy. LB v2: LongBench v2, LCB: LiveCodeBench, LME: LongMemEval.

Model	Method (budget)	Reasoning			Agentic	
		LB v2	AIME	LCB	τ^2-bench	LME
Qwen3-8B	BF16	35.4	56.7	66.5	30.7	42.8
	NVFP4	33.3	50.0	49.2	17.5	26.0
	FP6^g	**35.4**	**56.7**	**61.4**	**30.7**	**39.9**
	AutoQuantize (4.5)	**35.4**	50.0	60.0	20.2	31.9
	DP-LLM† (4.5)	21.6 (4.8)	16.7 (4.5)	26.6 (4.6)	10.6 (4.5)	18.5 (4.5)
	AutoEMatic	**35.4** (4.6)	**56.7** (4.7)	60.5 (4.8)	27.2 (4.6)	38.2 (5.1)
Qwen3.8-27B	BF16	56.9	93.3	76.3	74.6	82.8
	NVFP4	48.7	**86.7**	70.6	50.0	70.6
	FP6^g	**56.9**	**86.7**	**76.3**	**71.7**	**82.8**
	AutoQuantize (4.8)	**56.9**	**86.7**	74.9	62.0	81.2
	DP-LLM† (4.8)	25.4 (4.7)	56.7 (4.8)	51.4 (4.5)	25.6 (4.6)	15.9 (4.7)
	AutoEMatic	**56.9** (4.5)	**86.7** (4.6)	**76.3** (4.6)	**71.7** (5.1)	**82.8** (4.8)
Gemma-4-26B	BF16	46.7	86.7	85.7	61.4	72.6
	NVFP4	40.2	63.3	75.3	46.5	63.6
	FP6^g	**45.5**	**80.0**	**83.1**	**60.5**	**70.1**
	AutoQuantize (5.0)	41.0	**80.0**	80.1	58.6	68.1
	DP-LLM† (5.0)	35.6 (4.8)	50.0 (5.4)	68.6 (5.4)	11.5 (4.9)	16.0 (4.9)
	AutoEMatic	**45.5** (5.0)	**80.0** (5.0)	**83.1** (5.0)	**60.5** (5.1)	**70.1** (4.9)



(a) Prefill (b) Decode

Figure 7. **Layer-wise precision assignment by AutoEMatic on MiniMax-M2.7.**

$4\delta_A$ in the subgroups it refines.

5 Accuracy and GPU Performance

5.1 Experimental Setup

Models, Datasets, and Baselines. We evaluate Qwen3-8B, Qwen3.8-27B, DeepSeek-R1-Llama-70B, Gemma-4-26B-A4B, and MiniMax-M2.7 on reasoning (LongBench v2 (Bai et al., 2025), AIME 2024 (MAA, 2024), LiveCodeBench (Jain et al., 2025)) and agentic (τ^2-bench (Barres et al., 2025), LongMemEval (Wu et al., 2024), SWE-bench Verified (OpenAI, 2024)) workloads. For WxAx-KVx quantization, we compare against uniformly quantized NVFP4 and FP6^g, AutoQuantize (NVIDIA ModelOpt, 2025), which selects between NVFP4 or FP6^g per layer under a bit budget, and DP-LLM (Kwon et al., 2026), which switches among 3- to 6-bit of an Any-Precision format (Park et al., 2024). Since DP-LLM adapts only weights, we extend its policy to activations and KV cache (DP-LLM†) for a fair comparison. AutoQuantize, DP-LLM†, and AutoEMatic's refinement-aware GPTQ for weights use 256 samples from WikiText-2 for calibration. All experiments run in vLLM (Kwon et al., 2023) on NVIDIA GB300 GPUs with a 32K-token generation limit at temperature 0.5.

Hyperparameters. We set $\mathcal{P} = \{2, 4, 6\}$; rules $n_p = 2$ and $\alpha = 4$, $\beta = 2$; entropy threshold $\tau = 0.10$, $\mathcal{G}_\mathcal{E}$ is computed over the top-$K = 10$ logits; subgroup size $SG = 4$; AutoFP2 scale multiplier $c = 2$; activation NMSE threshold 0.003; weight refinement budget $\rho = 0.2$; KV page size $T = 64$. Hyperparameters are ablated in Appx.

Storage and Execution Precision. *Storage precision* is the packed model size in average bits per weight including metadata. For AutoQuantize, it is the target bit budget plus 0.5-bits for scale, 6-bits for DP-LLM† and $4 + 0.5 + 2\rho = 4.9$ bits for AutoEMatic at $\rho = 0.2$ (since the masks are shared across rows for weights, it is negligible). *Execution precision* is the nominal bitwidth at which layers execute averaged over layers and inference steps.

5.2 Main Accuracy Results

Comparison with Static and Runtime Baselines. AutoEMatic operates without a bit budget: its execution precision follows the dynamic decisions of the runtime policy. For a fair comparison, AutoQuantize and DP-LLM† are calibrated to the budget nearest AutoEMatic's achieved average. As shown in Table 2, across models and datasets, at 4.5–5.1 bits of average execution precision, accuracy of AutoEMatic exceeds AutoQuantize by up to 10% and DP-LLM† by up to 66.9% under a similar budget. AutoQuantize has a high underprovision rate which directly impacts its accuracy. AutoEMatic's advantage is particularly pronounced in agentic tasks, whose repeated prefills and long decodes impose dynamic precision demands, that DP-LLM†'s offline-calibrated runtime-policy cannot reliably track. DP-LLM† is further penalized by its nested integer format: the sub-6-bit precisions extracted from the shared representation incur substantial error. Overall, through runtime adaptation driven by model-internal signals and the AutoFP format, **AutoEMatic improves accuracy by up to 21.7% over NVFP4** and matches FP6^g, with a storage precision of 4.9 bits.

Large-Scale Agentic Analysis. On MiniMax-M2.7, a 230B-parameter agentic coding model, evaluated on SWE-bench Verified, AutoEMatic matches FP6^g accuracy and remains within 0.5% of baseline (at FP8) at an average execution precision of 4.2 bits in prefill and 4.5 in decode, against 6 bits for FP6^g (Table 3). Figure 7 demonstrates AutoEMatic's runtime adaptation across prefill and decode.

Table 3. **Accuracy comparison of MiniMax-M2.7 on SWE-bench Verified.**

Method	Exec. Precision		Acc. (%)
	Prefill	Decode	
FP8	8	8	71.0
NVFP4	4	4	67.8
FP6^g	6	6	70.5
AutoEMatic	4.2	4.5	**70.5**

5.3 GPU Performance Analysis

Figure 8 reports prefill time-to-first-token (TTFT) and decode throughput for DS-Llama-70B and MiniMax-M2.7 on 4× GB300 GPUs, sweeping input lengths of 1K–128K tokens for prefill and batch sizes of 1–128 for decode with

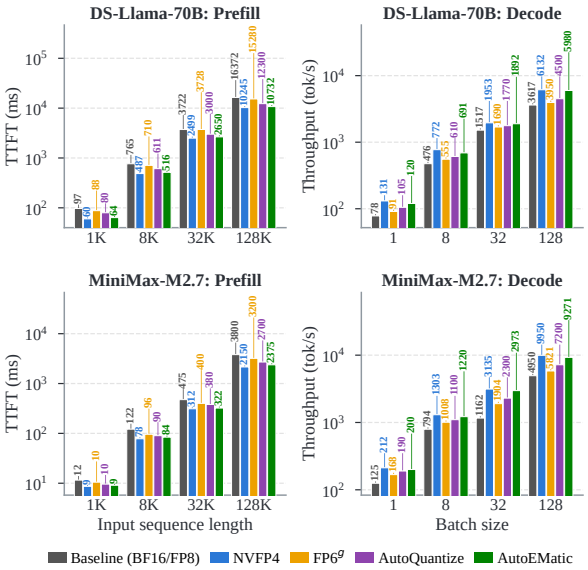


Figure 8. **Decode throughput and prefill TTFT analysis on 4× GB300 GPUs for DS-LLaMA-70B and MiniMax-M2.7.**

an 8K-token input and a 16K-token output. We compare against methods that execute on Tensor Cores: NVFP4 through the native CUTLASS W4A4 path, FP6^{g} through a fused kernel that dequantizes E2M3 to FP8, and AutoQuantize uses either based on NVFP4/FP6^{g} layers. AutoEMatic's runs use precision assignments from LongBench v2.

Prefill. AutoEMatic reduces TTFT by up to 1.4× over FP6^{g} and 1.25× over AutoQuantize across input lengths. Specifically, despite AutoEMatic's runtime precision adaptation enabling superior accuracy compared to NVFP4, the TTFT is within 10% of the NVFP4 baseline (small performance penalty). Two factors keep this performance penalty small: (a) when all requests in a batch choose 4-bit for a layer, they execute at NVFP4 throughput and (b) when any request chooses AutoFP6 it executes as a single extended-K GEMM on Tensor Cores, instead of separate GEMMs (§7).

Decode. AutoEMatic improves decode throughput by 1.5–2.5× over the full-precision baseline across batch sizes, and attains 90–98% of NVFP4 throughput, i.e., near-NVFP4 throughput at FP6^{g} accuracy.

6 Accelerator Architecture

6.1 Architecture Overview

On GPUs, AutoFP2 is emulated and AutoFP6 requires an extension of the reduction dimension. An AutoFP-native datapath targets both limitations of existing hardware. We extend a representative accelerator MAGNet (Venkatesan et al., 2019; Hooper et al., 2025) as proof-of-concept. The output-stationary local-weight-stationary (OS-LWS) design (Figure 9(a)) has 256 PEs and a global SRAM (L2) that loads data from DRAM (modeled). Each PE (Figure 9(b)) has mixed-precision vector MAC (vMAC) that computes one

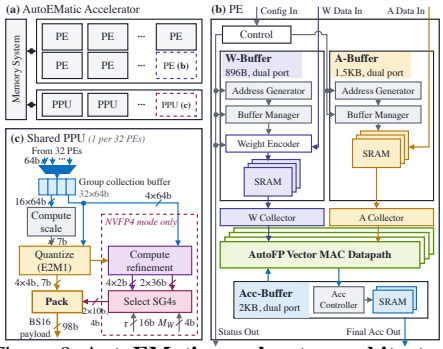


Figure 9. **AutoEMatic accelerator architecture.**

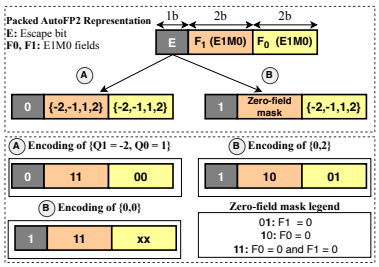


Figure 10. **AutoFP2 representation.**

V-element NVFP4/AutoFP6 or two V-element AutoFP2 dot products per cycle. A PPU processes completed outputs (Figure 9(c)). Signals `mode_nvfp4` and `refine_en` select AutoFP2, NVFP4, or AutoFP6 modes of operation.

6.2 Weight Encoder and Packed AutoFP2 Format

Weights are stored in DRAM as NVFP4 (§3.2), but inside the PEs every weight is held in AutoFP2, so that the vMAC can execute all modes with AutoFP2 arithmetic units. Two requirements follow. First, for the vMAC to process AutoFP2 at twice the throughput of NVFP4, each weight load from the buffers must deliver two AutoFP2 weights. Second, in NVFP4 mode a single AutoFP2 digit cannot represent an NVFP4 value. We introduce a packed AutoFP2 format that satisfies both by encoding a pair of AutoFP2 digits (Q_1, Q_0) in 5 bits (2.5 bits per digit). In the packed format (Figure 10), an escape bit E precedes two 2-bit fields F_1 and F_0. A nonzero AutoFP2 digit is one of ± 1 and ± 2. When both digits are nonzero ($E = 0$), F_1 and F_0 hold them in E1M0. Otherwise ($E = 1$), F_1 records which digits are zero and F_0 holds the nonzero digit, if any. The weight encoder forms the AutoFP2 packed format as weights enter the PE and writes them to the weight buffer. The packing overhead is amortized by the weight stationary dataflow. In AutoFP2 mode (`mode_nvfp4=0`), it applies the exponent remapping of §3.2 to two weights from adjacent rows (different K dimension) that share an input activation and packs the two digits. In NVFP4/AutoFP6 modes (`mode_nvfp4=1`), the pair represents one NVFP4 weight as $W_q = c_0 Q_0 + c_1 Q_1$ with fixed coefficients. We set $W_q = (5Q_0 + 2Q_1)/2$, such that their linear combinations spans the maximum NVFP4 magnitude 6 (derivation

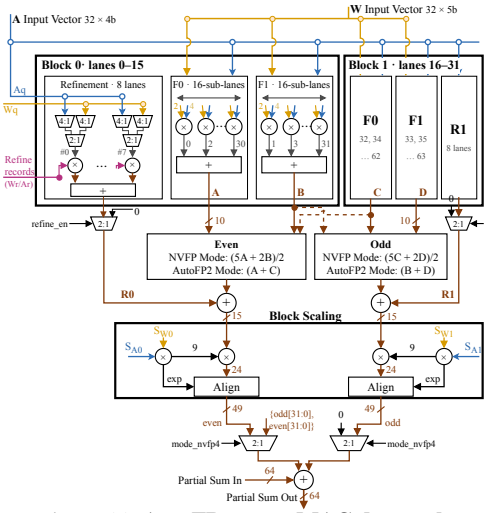


Figure 11. **AutoFP vector MAC datapath.**

in Appx.). For e.g., $0.5 = (5{\cdot}1{+}2{\cdot}(-2))/2$. Each AutoFP2 digit is a Boolean function of the E2M1 bits (§D.7).

6.3 vMAC Datapath Design

Each PE is a $V = 32$ element wide vMAC (Figure 11) organized as two $V/2 = 16$ base lane blocks each with dedicated set of refinement lanes and pipelined into multiply and accumulate stages. The vMAC accepts weights in the packed AutoFP2 format. The refinement data has a separate input interface carrying the masks and signed offsets of the refined subgroups. Each base lane k comprises an even ($F_{0,k}$) and an odd ($F_{1,k}$) AutoFP2 sub-lane that shares one streamed NVFP4 activation. For energy-efficiency, all computations use fixed-point arithmetic (§D.8).

Base Lanes. In the multiply stage, the escape bit E of each packed pair decodes the two digits. When $E = 1$, F_1 identifies which digit is zero, and the corresponding sub-lane is bypassed: its product is set to zero. For a nonzero digit, F_1 or F_0 holds an E1M0 code, ±1 or ±2, so each sub-lane forms $A_k F_{j,k}$, $j \in \{0, 1\}$, with simple shifters. Within a block, the even and odd sub-lanes feed separate adder trees in the adder stage, $A = \sum_{k=0}^{15} A_k F_{0,k}$ and $B = \sum_{k=0}^{15} A_k F_{1,k}$ for lanes 0–15, and C, D for lanes 16–31. Every execution mode differs only in how the tree outputs are combined. In NVFP4 mode, the A, B correspond to the partial sum accumulated from the same weight $W_{q,k}$, therefore the mode-dependent reduction unit computes $P0 = (5A + 2B)/2$ (implemented in fixed-point) to form the corresponding NVFP4 partial sum, and similarly for P_1, as shown in Figure 11. The base lanes perform the same operation in AutoFP6 mode. In AutoFP2 mode, within a block the two sub-lanes process corresponding weights from different rows (so that they can share one input activation). The weights of the same row are therefore spread across the even and odd sub-lanes of both blocks, so the mode-dependent reduction unit sums across blocks,

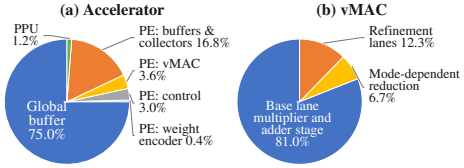


Figure 12. **Area breakdown of (a) the AutoEMatic accelerator and (b) its vMAC.**

$P_0 = A + C$ and $P_1 = B + D$.

Sparse Refinement Lanes. The refinement datapath adds $V/4 = 8$ refinement lanes per block as two $SG = 4$-lane slots (in §7 we establish this as sufficient). In AutoFP2/NVFP4 mode (`refine_en=0`), the refinement lanes are gated for energy efficiency. Because $\mathbf{M}_W$ and $\mathbf{M}_A$ are disjoint, a refined weight/activation subgroup occupies exactly one slot. The collector supplies each slot with the refinement mask, four 2-bit signed-offsets d, and a bit identifying weight/activation refinement. Each lane has two 4:1 muxes that pick the corresponding base-weight and activation subgroups using the mask; a 2:1 selector chooses the corresponding X_q: A_q for weight refinement, or W_q for activation refinement (note the similarity with construction of W_{qr}, A_{qr}). Each lane forms $R = 2X_q d$ in fixed-point arithmetic. Each refinement lanes' contributions are accumulated as $\mathcal{R} = \sum R$. It is accumulated with the base lanes partial sums as, $O_j = 4P_j + \mathcal{R}$.

Scaling and accumulation. Each O_j is multiplied by the scale factor of its corresponding block. In AutoFP2 mode, each O_j is a 32-element dot product corresponding to two different weight rows with the same activations. In NVFP4 and AutoFP6 mode, each O_j corresponds to a 16-element dot product along the same row (same K dimension). After scaling, O_0 and O_1 are summed into one 64-bit accumulator in NVFP4/AutoFP6, while in AutoFP2 mode O_0, O_1 are held in the two 32-bit halves of the accumulator.

6.4 Shared Post-Processing Unit

The PPU (Figure 9(c)) quantizes completed PE outputs into the next layer's input activations. If a layer requires refinement, it also computes per-subgroup NMSE and selects subgroups with NMSE above τ excluding those in $\mathbf{M}_W$ and assigns signed offsets d. The PPU also performs non-linear operations, and computes the metric $\mathcal{M}_{P/D}$.

6.5 Accelerator Evaluation

Implementation. The accelerator is implemented in SystemC, with RTL generated using Siemens Catapult HLS. We use a 2MB global SRAM, 896B weight, 1.5KB activation, and 2KB accumulation buffers. We perform synthesis in TSMC 3nm at a frequency of 1 GHz. Synopsys VCS is used for simulation and PrimeTime PX for power estimation.

Area Analysis. Figure 12(a) breaks down the area of the AutoEMatic accelerator. The 2MB global SRAM occupies

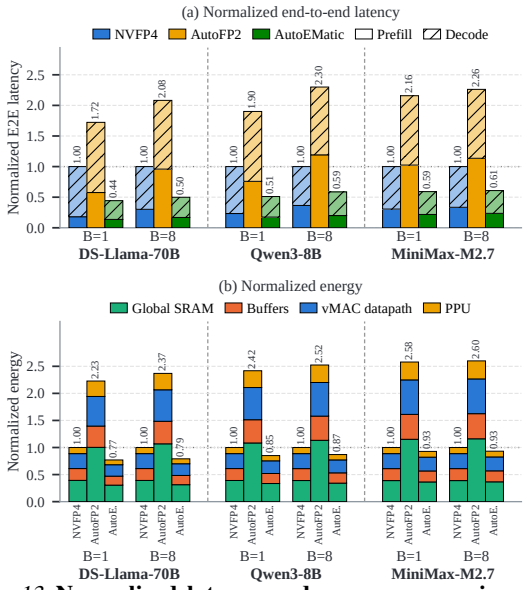


Figure 13. **Normalized latency and energy comparison with single-format baselines, under iso-accuracy.**

Table 4. **Component ablations** with DS-LLaMA-70B on LCB.

Component	Bits	Acc.	2 / 4 / 6 (%)
1. BF16	16	68.4	–
2. NVFP4 (W-A-KV)	4.5	51.7	0 / 100 / 0
3. + AutoEMatic runtime policy w/ $\mathcal{P} = \{2, 4, 6\}$ copies	13.5	66.2	20 / 40 / 40
4. AnyPrecision LLM on $\mathcal{P}$ + AutoEMatic runtime policy	6	43.8	8 / 42 / 50
5. AutoFP6	4.9	64.8	0 / 0 / 100
6. + Refinement-aware GPTQ	4.9	65.6	0 / 0 / 100
7. **AutoEMatic format + runtime policy**, $\mathcal{P} = \{2, 4, 6\}$	4.9	**65.6**	15 / 45 / 40

75% of the area. The PE array, with all 256 PEs, takes 23.8%. The per-PE weight encoders account for only 0.4% and the vMAC 3.6% of the total area; within a vMAC the refinement lanes (12.3%) and the mode-dependent reduction logic for NVFP4- and AutoFP2-mode (6.7%), result in a net 1.2× overhead over an NVFP4-only vMAC. The accelerator is only 1.1% more area than the NVFP4 baseline.

Iso-Accuracy Performance Comparison. We compare against single-format AutoFP2 and NVFP4 baselines under iso-accuracy with AutoEMatic by replaying its layer-wise precision traces, as recorded on LiveCodeBench. Layers above a baseline's native precision execute with an extended reduction dimension, accumulating into the same partial sums; the NVFP4 design, for instance, executes a refined layer analogous to the extended-K GEMM (§4). Figure 13(a) reports end-to-end latency normalized to the NVFP4 design at batch sizes 1 and 8. AutoEMatic is 1.6–2.3× faster than NVFP4 for two reasons: AutoFP2 layers run at twice the NVFP4 throughput, which benefits compute-bound prefill most (1.3–1.8× faster), and refined layers execute natively in the sparse refinement lanes without the additional pass an extended-K reduction requires. Across evaluated models, on average, AutoEMatic achieves a compute density of 5.46 TOPS/mm^2, compared to 3.23 TOPS/mm^2 for NVFP4 and 6.16 TOPS/mm^2 for AutoFP2.

Iso-Accuracy Energy Comparison. Figure 13(b) reports energy for the same traces, normalized to NVFP4 and broken down into different components, each including static and dynamic energy. AutoEMatic consumes 0.77–0.93× the energy of NVFP4, as (a) all modes use AutoFP2 lanes and (b) lower runtime. Its vMAC datapath averages 15fJ/op, against 22fJ/op for NVFP4 and 10fJ/op for AutoFP2.

7 Discussion and Ablations[3]

[Algorithm+Quantization] Component Ablations. Table 4 isolates the runtime policy and the quantization format. Applying AutoEMatic's runtime policy to separately quantized AutoFP2, NVFP4, and FP6^g copies (row 3), attains 66.2%, but needs 13.5 stored bits per weight. Applied to the integer multi-scale Any-Precision format (row 4), attains only 43.8%. This suggests that a favorable accuracy–efficiency tradeoff requires both: the runtime policy and an FP-based multi-scale format. Combined (row 7), AutoEMatic preserves 65.6%, within 0.6% of the separate-copy upper bound, at 4.9 bits, executing 40% layers at 6-bit, 45% at 4-bit and 15% at 2-bit. Selecting the refined weight subgroups by NMSE alone, with a threshold of 0.003 at the same budget ρ (Table 4, row 5), attains 64.8%; refinement-aware GPTQ (row 6), which selects them by their reduction in the Hessian-weighted error raises accuracy to 65.6%.

[Quantization] Refinement Decoding. Figure 14(a) compares decoding signed offsets with the fixed step 2^{-3} against exact, per-element exponent computation 2^{e-3} on DS-LLaMA-70B and Qwen3-8B on LCB. The fixed-step has similar accuracy as the exact exponent computation, while achieving up to 28% higher decode throughput. This is because the fixed-step does not require loading the base value when constructing the extended-K GEMM operands.

[Algorithm] AutoEMatic Runtime Policy vs. Oracle. Figure 14(b) compares per-layer underprovision rate of AutoEMatic's precision decisions against the oracle of §2 restricted to the precision set $\mathcal{P}$. AutoEMatic underprovisions only 7.7% of layer executions on average, and its 2/4/6-bit distribution of 15/45/40% closely tracks the oracle's 20/41/39%. Using $\mathcal{L}_{\mathcal{A}}$ alone has a higher rate of underprovisioning, demonstrating the combined metric $\mathcal{M}_D$ as the optimal choice. We omit $\mathcal{G}_{\mathcal{E}}$ from this comparison because it is a global signal shared across layers.

[Algorithm] Rule-based vs. Instantaneous Adaptation. Figure 14(g) traces one decode step of DS-Qwen-14B layer by layer: $\mathcal{M}_D$ (top) and the precision assigned by the oracle of §2.1, by an instantaneous policy that switches with every rise and fall in $\mathcal{M}_D$, and by AutoEMatic's rule-based policy. Evident from the oracle's own behavior, the rules are not ad hoc, oracle precision does not rise (drop) immediately after an increase (decrease) in the metric, but only once

[3] See Appx. for additional discussions and ablations.

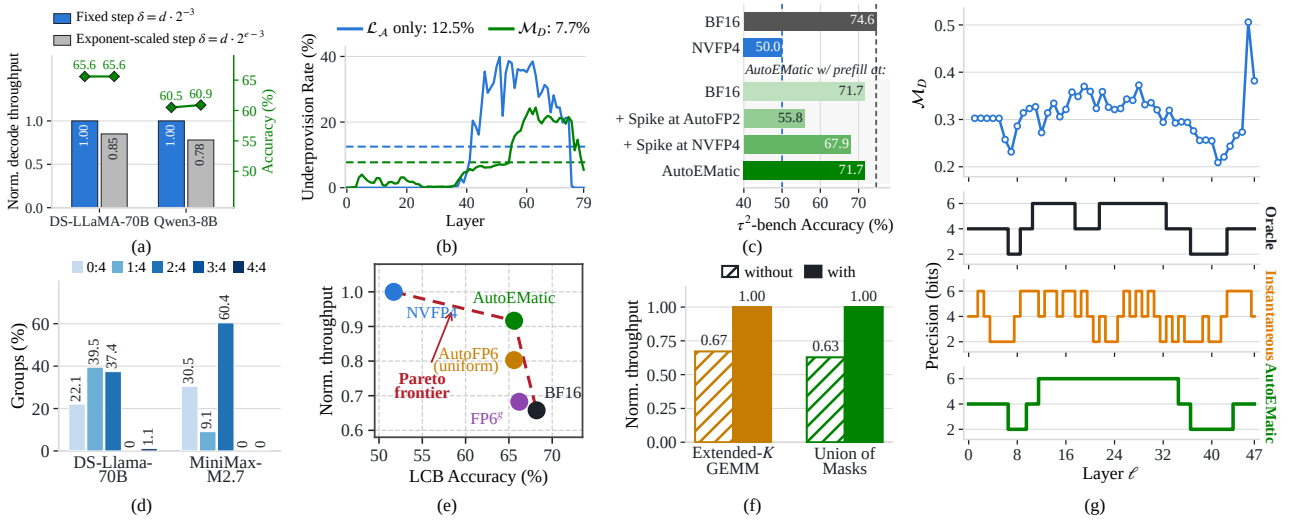


Figure 14. **Ablations. (a)** Fixed vs. exponent-scaled decoding of d. **(b)** Underprovision rate of AutoEMatic. **(c)** Prefill precision at spike. **(d)** Refined subgroups per group. **(e)** Accuracy vs. throughput pareto. **(f)** Kernel ablations. **(g)** For one decode step: $\mathcal{M}_D$ across layers and the precision assigned by the oracle, an instantaneous rule, and AutoEMatic.

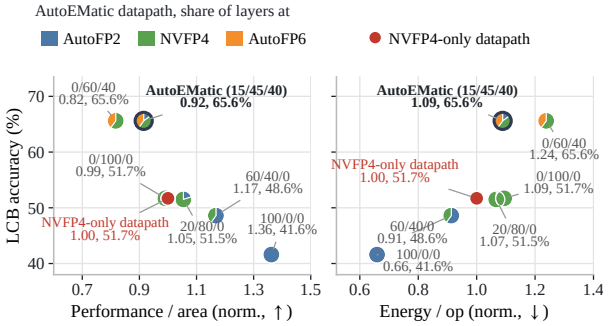


Figure 15. **DS-LLaMA-70B LCB accuracy vs. performance per area and energy per op**, normalized to an NVFP4-only datapath.

the change persists over several layers. The instantaneous policy switches on blips and drops to 2 bits at layers where the oracle requires 6. AutoEMatic's rules are built inspired by this behavior of the oracle and change precision only once the model has provided sufficient evidence.

[Algorithm] Prefill Precision at the Spike. Figure 14(c) varies only how the spiking layer in prefill is executed, keeping the decode policy fixed. Running prefill entirely in BF16 gives 71.7%, the same as AutoEMatic, which executes the spike layers at 6 bits. Lowering the spike layer to NVFP4 costs 3.8% and to AutoFP2 15.9%.

[Quantization] Subgroups Refined Per Group. Figure 14(d) shows the distribution of weight+activation subgroups refined per group, $N_W + N_A$, for DS-LLaMA-70B on LCB and MiniMax-M2.7 on SWE-bench. 22–31% of groups need no refinement (0:4), and only 1.1% of DS-LLaMA-70B groups and none in MiniMax-M2.7 exceed 2:4, supporting the choice of 8 refinement lanes.

[System] Kernel Ablations. Figure 14(e) plots accuracy against decode throughput normalized to NVFP4 for DS-LLaMA-70B on LCB at batch 512. AutoEMatic lies on the Pareto frontier: 1.3× faster than $FP6^g$, and 1.1× faster than uniform AutoFP6. AutoEMatic's superior performance-accuracy tradeoff stems from the runtime policy: it assigns 4 bits to most layers, which then run at NVFP4 throughput, and 6 bits only where accuracy requires it, which still runs as a single extended-K GEMM on the same NVFP4 Tensor Cores. Figure 14(f) ablates AutoEMatic's two kernel mechanisms with different batch sizes. Without the extended-K GEMM, executing the base and refinement GEMMs separately and summing their outputs lowers throughput by 1.5× at batch size 1; without the union of masks, splitting mixed-precision batches into per-precision sub-batches lowers throughput by 1.6× at batch size 512.

[Accelerator] Precision Mix Sweep. Figure 15 evaluates different layer-wise precision distributions on DS-LLaMA-70B against an NVFP4-only datapath. AutoEMatic's runtime assignment (15/45/40) lies on the Pareto frontiers of accuracy versus area efficiency and accuracy versus energy efficiency among the evaluated configurations of different precision distributions, achieving 65.6% accuracy with only an 8% performance/area penalty and 9% higher energy/op.

8 Related Work

Multi-scale quantization. Any-Precision LLM (Park et al., 2024), Matryoshka Quantization (Nair et al., 2025), Any-BCQ (Park et al., 2026), and RRQ (Luo et al., 2026b) store several precisions within one integer representation, as nested bit planes or successive residual stages, from which the selected precision is reassembled before computation.

Runtime precision adaptation. FlexQuant (Liu et al.,

2025), DP-LLM (Kwon et al., 2026), MoBiQuant (Wang et al., 2026), and QuickSilver (Khanna et al., 2025) adapt weight precision during decoding through predetermined schedules, calibrated error thresholds, or trained routers, and Mix-Quant (Lu et al., 2026) assigns NVFP4 to prefill and BF16 to decoding. For the KV cache, PM-KVQ (Liu et al., 2026) progressively lowers precision and ThinKV (Ramachandran et al., 2026) assigns precision by reasoning importance. Appx. provides an extended discussion.

9 Conclusion

AutoEMatic shows that adapting precision to the evolving numerical demands of LLM inference improves its accuracy–efficiency tradeoff. AutoFP switches precision at runtime through mantissa refinement and exponent remapping of a shared NVFP4 base, without requiring redundant weight copies. Future work will explore achieving memory benefits with AutoFP2 and extending AutoFP to higher precisions through joint exponent and mantissa refinement.

References

Alvarez, E. et al. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, June 2025. Accessed September 19, 2026.

Bai, Y., Tu, S., Zhang, J., Peng, H., Wang, X., Lv, X., Cao, S., Xu, J., Hou, L., Dong, Y., et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3639–3664, 2025.

Barres, V., Dong, H., Ray, S., Si, X., and Narasimhan, K. τ^2-bench: Evaluating conversational agents in a dual-control environment, 2025.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

Hayashi, S.-i., Mukunoki, D., Hoshino, T., and Katagiri, T. Dgemm with ozaki scheme i/ii on fp4 tensor cores: A base-13 e2m1 limb representation. *arXiv preprint arXiv:2608.06812*, 2026.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Hooper, C., Sakr, C., Keller, B., Venkatesan, R., Keutzer, K., Shao, S., and Khailany, B. Fgmp: fine-grained mixed-precision weight and activation quantization for hardware-accelerated llm inference. *arXiv preprint arXiv:2504.14152*, 2025.

Hu, W., Zhang, Z., Zhang, H., Zhang, C., Guo, C., Feng, Y., Hu, T., Li, G., Hu, G., Wang, J., and Leng, J. M^{2}XFP: A metadata-augmented microscaling data format for efficient low-bit quantization. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 1151–1167, 2026. doi: 10.1145/3779212.3790185. URL https://arxiv.org/abs/2601.19213.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *International Conference on Learning Representations*, 2025.

Kariyappa, S. and Suh, G. E. Sidequest: Model-driven kv cache management for long-horizon agentic reasoning. *arXiv preprint arXiv:2602.22603*, 2026.

Khanna, D., Guru, A. K., Sridhar, S., Ahmed, Z., Bahirwani, R., Malhotra, M., Jain, V., Chadha, A., Das, A., and Ghosh, K. QuickSilver – speeding up LLM inference through dynamic token halting, KV skipping, contextual token fusion, and adaptive matryoshka quantization, 2025. URL https://arxiv.org/abs/2506.22396.

Kwon, S., Seo, S. H., Lee, J. W., and Park, Y. Dp-llm: Runtime model adaptation with dynamic layer-wise precision assignment. *Advances in Neural Information Processing Systems*, 38:144921–144945, 2026.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Lee, J., Park, J., Cha, S., Cho, J., and Sim, J. MX+: Pushing the limits of microscaling formats for efficient large language model serving. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, pp. 869–883, 2025. doi: 10.1145/3725843.3756118. URL https://arxiv.org/abs/2510.14557.

Lee, M., Ramachandran, A., Kundu, S., and Krishna, T. Polestar: Drift-aware cache calibration and token commitment for efficient inference of diffusion llms. *arXiv preprint arXiv:2607.14107*, 2026a.

Lee, S., Lee, J., Yoo, D., Kim, J. G., Ryu, H., Ryu, S., and Choi, J. Reset: Accurate latency-critical nvfp4 reasoning via step-aware temperature scaling. *arXiv preprint arXiv:2606.13233*, 2026b.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 87–100, 2024.

Liu, F., Wang, Z., Xia, J., Zhao, J., Zhao, S., Li, J., Liu, J., Jiang, L., and Guan, H. Flexquant: A flexible and efficient dynamic precision switching framework for llm quantization. *arXiv preprint arXiv:2506.12024*, 2025.

Liu, T., Li, S., Yang, J., Zhao, T., Zhou, F., Song, X., Dai, G., Yan, S., Yang, H., and Wang, Y. PM-KVQ: Progressive mixed-precision KV cache quantization for long-CoT LLMs. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2505.18610.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning*, 2024.

Lu, H., Chen, Z., Fang, G., Ma, X., and Wang, X. Mixquant: Quantized prefilling, precise decoding for agentic llms. *arXiv preprint arXiv:2605.20315*, 2026.

Luo, J., Jiang, X., Wen, T.-H., Qi, R., Chu, M., Wu, H., Kielian, G., Laurie, B., Zhang, Q., Cheng, Q., Sylvester, D., and Saligane, M. Heterogeneity-aware microscaling for efficient low-bit LLM inference, 2026a. URL https://arxiv.org/abs/2608.03867.

Luo, Y., Dong, B., Cheng, W., and Shen, H. Recurrent residual quantization: A progressive multi-precision representation for LLMs, 2026b. URL https://arxiv.org/abs/2608.04048.

MAA. Aime 2024 problems. https://artofproblemsolving.com/wiki/index.php/2024_AIME_I_Problems, 2024. Accessed: 2025-08-30.

Nair, P., Datta, P., Dean, J., Jain, P., and Kusupati, A. Matryoshka quantization. *arXiv preprint arXiv:2502.06786*, 2025.

NVIDIA ModelOpt. NVIDIA Model Optimizer–AutoQuantize. GitHub repository, 2025. URL https://github.com/NVIDIA/Model-Optimizer.

Open Compute Project. OCP microscaling formats (MX) specification. Technical Report Version 1.0, Open Compute Project Foundation, 2023.

OpenAI. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/, August 2024. Accessed 2026-10-12.

Park, G., Bae, J., Kwon, B., Kim, B., Kwon, S. J., and Lee, D. Anybcq: Hardware efficient flexible binary-coded quantization for multi-precision llms. In *International Conference on Learning Representations*, volume 2026, pp. 125106–125122, 2026.

Park, Y., Hyun, J., Cho, S., Sim, B., and Lee, J. W. Any-precision llm: Low-cost deployment of multiple, different-sized llms. *arXiv preprint arXiv:2402.10517*, 2024.

Ramachandran, A., Kundu, S., and Krishna, T. Clamp-vit: Contrastive data-free learning for adaptive post-training quantization of vits. In *European Conference on Computer Vision*, pp. 307–325. Springer, 2024.

Ramachandran, A., Kundu, S., and Krishna, T. Microscopiq: Accelerating foundational models through outlier-aware microscaling quantization. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pp. 1193–1209, 2025a.

Ramachandran, A., Kundu, S., Raha, A., Kundu, S., Mathaikutty, D. K., and Krishna, T. Accelerating llm inference with flexible n: M sparsity via a fully digital compute-in-memory accelerator. In *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, pp. 1–7. IEEE, 2025b.

Ramachandran, A., Neseem, M., Sakr, C., Venkatesan, R., Khailany, B., and Krishna, T. Thinkv: Thought-adaptive kv cache compression for efficient reasoning models. In *International Conference on Learning Representations*, volume 2026, pp. 110072–110106, 2026.

Rouhani, B. D., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

Shen, Y. and Zou, A. River-llm: Large language model seamless exit based on kv share. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 37633–37645, 2026.

Venkatesan, R., Shao, Y. S., Wang, M., Clemons, J., Dai, S., Fojtik, M., Keller, B., Klinefelter, A., Pinckney, N., Raina, P., et al. Magnet: A modular accelerator generator for neural networks. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 1–8. IEEE, 2019.

Wang, D., Kim, J., Han, S., Gudovskiy, D., Nakata, Y., Okuno, T., Peong, K., Jeon, K. E., Ko, J. H., Chen, Y., et al. Mobiquant: Mixture-of-bits quantization for token-adaptive any-precision llm. *arXiv preprint arXiv:2602.20191*, 2026.

Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

Zhao, Z., Wang, M., Yu, B., Liu, B., Han, X., and Zhao, X. DynamicPTQ: Mitigating activation quantization collapse via residual-stream dynamics, 2026. URL https://arxiv.org/abs/2606.12487.

Zou, J., Chen, Y., Wu, R., and Chen, X. MixFP4: Enhancing NVFP4 with adaptive FP4/INT4 block representations. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://arxiv.org/abs/2605.31035.

APPENDIX

A Extended Related Work **14**

B Low-Precision Floating-Point Formats **15**

B.1 NVFP4 15

B.2 FP6^{g} 15

B.3 Two-Bit Formats and the AutoFP2 Digit . . 15

B.4 Sharing NVFP4 Scales with AutoFP2 . . . 16

B.5 Worked Example of Mantissa Refinement . 16

C Additional Motivating Analyses **16**

C.1 Layer-Wise Precision Across Decode Steps 16

C.2 Layer-Wise Precision Across Prefill and Refills 16

C.3 Precision Demand by Generation Entropy . 16

C.4 Layer-Wise Behavior of $\mathcal{M}_P$ and $\mathcal{M}_D$ Across Models and Datasets 16

C.5 Subgroup Refinement on Additional Models 17

D Additional AutoEMatic Details **17**

D.1 Signals Used for Runtime Precision Adaptation 17

D.2 Entropy Signal Across Decode Steps 17

D.3 Memory Layout 17

D.4 Extended-K GEMM 17

D.5 Construction of the Extended-K Operands . 17

D.6 Numerical Example of the Extended-K GEMM 17

D.7 NVFP4-to-AutoFP2 Decomposition 17

D.8 Fixed-Point Arithmetic of the vMAC 18

D.9 Accelerator Floorplan 18

E Refinement-Aware GPTQ Calibration **19**

F GPU Implementation Details **19**

G Accelerator Simulation Details **19**

H Additional Accuracy Results **19**

I Additional Ablations **19**

J Limitations **23**

K AI Usage Statement **23**

A EXTENDED RELATED WORK

Static post-training quantization. Most PTQ methods choose a precision offline and keep it fixed. GPTQ (Frantar et al., 2023) quantizes a weight matrix one column at a time and uses approximate inverse-Hessian information to update the remaining weights, compensating the quantization error. AWQ (Lin et al., 2024) identifies salient weight channels from activation statistics and protects them by per-channel scaling before quantization.

Group-scaled floating-point formats and their extensions. MX formats (Rouhani et al., 2023; Open Compute Project, 2023) share a power-of-two E8M0 scale across a group of 32 FP8, FP6, FP4, or INT8 elements, and NVFP4 (Alvarez et al., 2025) stores E2M1 elements in groups of 16 with an E4M3 group scale and a per-tensor FP32 scale. MX+ (Lee et al., 2025) observes that the group's largest element does not need its exponent field given the shared scale and repurposes those bits as extra mantissa for that element. M^{2}XFP (Hu et al., 2026) adds two metadata bits per subgroup, which serve as extra mantissa bits for the subgroup's largest activation and as a scale multiplier for weights. AdaMX (Luo et al., 2026a) chooses per group between FP4 and INT4 grids combined with group-maximum mantissa extension or scale refinement at a constant bit width. These extensions are fixed after quantization and refine one element or one scale per group. AutoEMatic is generally applicable to any group-scaled floating-point format.

Multi-scale quantization. Multi-scale methods store several precisions in one representation so that precision can change without separate checkpoints. Any-Precision LLM (Park et al., 2024) quantizes a seed model at the lowest bit width with non-uniform clustering, then adds one bit at a time by splitting each cluster, storing the codes as bit planes so that lower-bit codes are prefixes of higher-bit codes. Matryoshka Quantization (Nair et al., 2025) co-trains a single INT8 model so that slicing its most significant bits yields usable INT4 and INT2 models. AnyBCQ (Park et al., 2026) builds 2- to 4-bit models on binary-coded quantization by freezing existing bit planes and adding residual planes with re-optimized scales, and RRQ (Luo et al., 2026b) represents weights as a 2-bit base plus successive 2-bit quantized residuals, so that 4-, 6-, and 8-bit precisions are sums of residual stages.

Runtime precision adaptation. Mix-Quant (Lu et al.,

Table 5. Low-precision floating-point formats.

Format	S/E/M	Magnitudes	Bits/elem.
E2M1 (NVFP4)	1/2/1	$\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$	4.5
E2M3 (FP6^{g})	1/2/3	0–7.5 in steps of $\frac{1}{8}, \frac{1}{4}, \frac{1}{2}$	6.5
E1M0	1/1/0	$\{1, 2\}$	2.5
E0M1	1/0/1	$\{0, 1\}$	2.5
Quinary digit (AutoFP2)	–	$\{0, 1, 2\}$	3.0
E4M3 (scale)	1/4/3	$2^{-9} - 448$	–

Table 6. Positive magnitudes of E2M1 (NVFP4) and E2M3 (FP6^{g}).

E\M	**E2M1 (NVFP4)**		**E2M3 (FP6^{g})**							
	0	1	000	001	010	011	100	101	110	111
00	0	0.5	0	0.125	0.25	0.375	0.5	0.625	0.75	0.875
01	1	1.5	1	1.125	1.25	1.375	1.5	1.625	1.75	1.875
10	2	3	2	2.25	2.5	2.75	3	3.25	3.5	3.75
11	4	6	4	4.5	5	5.5	6	6.5	7	7.5

2026) applies NVFP4 to prefill and keeps BF16 for decoding, fixing each phase's precision. FlexQuant (Liu et al., 2025) stores both INT8 and INT4 weights and, during decoding, replaces INT8 layers with INT4 in an order fixed offline by KL-divergence analysis whenever a moving average of output entropy falls below a threshold. MoBiQuant (Wang et al., 2026) decomposes weights into recursive 2-bit residual slices and uses a trained token router to decide how many slices each token activates. QuickSilver (Khanna et al., 2025) computes, at a layer, the entropy of each token's hidden representation and assigns 8-, 4-, or 2-bit precision by thresholds.

KV-cache compression. KIVI (Liu et al., 2024) quantizes the key cache per channel and the value cache per token to 2 bits without tuning, and KVQuant (Hooper et al., 2024) combines per-channel pre-RoPE key quantization, sensitivity-weighted non-uniform datatypes, and a dense-and-sparse split that keeps outliers separately. For long chain-of-thought reasoning, PM-KVQ (Liu et al., 2026) lowers the bit width of the KV cache progressively as the context grows and allocates higher bit widths to more sensitive blocks, and ThinKV (Ramachandran et al., 2026) classifies chain-of-thought segments into thought types from attention sparsity, assigning token precision by thought importance and evicting tokens of less critical thoughts.

Mixed-precision accelerators. MicroScopiQ (Ramachandran et al., 2025a) retains outlier weights at higher precision and offsets their additional bits by pruning the least important weights, executing on multi-precision integer processing elements with a network-on-chip that routes the high-precision outliers. AdaMX (Luo et al., 2026a) shares one multiplier datapath across its group formats and applies up to three per-group scalar correction terms through an auxiliary path. MixFP4 (Zou et al., 2026) selects one of two FP4 element formats, E2M1 or E1M2, per group, signaled by the sign bit of the E4M3 group scale, and decodes both to a common internal E2M2 representation.

B Low-Precision Floating-Point Formats

We write EeMm for a signed floating-point format with one sign bit, e exponent bits, and m mantissa bits. Table 5 summarizes them.

B.1 NVFP4

NVFP4 (Alvarez et al., 2025) stores each element as an E2M1 value (exponent bias 1) with magnitudes $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$ under two levels of scaling. A tensor X is split into groups X_j of 16 contiguous elements along the reduction dimension; each group has an E4M3 *group scale* B_j, and the tensor has an FP32 *tensor scale* T:

$$T = \frac{\operatorname{amax}(X)}{6 \cdot 448}, \qquad B_j = Q_{\mathrm{E4M3}}\left(\frac{\operatorname{amax}(X_j)}{6\,T}\right), \tag{2}$$

$$q_i = Q_{\mathrm{E2M1}}\left(\frac{x_i}{T B_j}\right), \quad \widehat{x}_i = T B_j\, q_i, \quad i \in X_j, \tag{3}$$

where Q_F rounds to the nearest value of format F (ties to even). The group scale maps each group's absolute maximum onto 6, the largest E2M1 value, and the tensor scale maps the largest group scale onto 448, the largest E4M3 value. With one 8-bit scale per 16 elements, NVFP4 costs $4 + 8/16 = 4.5$ bits per element.

B.2 FP6^{g}

FP6^{g}, the group-scaled 6-bit counterpart of NVFP4 used as a baseline throughout the paper, keeps the group size and scaling hierarchy of NVFP4 and replaces E2M1 by E2M3 (exponent bias 1). The two additional mantissa bits give 32 magnitudes, in steps of $\frac{1}{8}$ on $[0, 2)$, $\frac{1}{4}$ on $[2, 4)$, and $\frac{1}{2}$ on $[4, 7.5]$, and lower the worst-case relative rounding error of normal values from $1/4$ to $1/16$. The scales follow eqs. (2) and (3) with the E2M3 maximum 7.5 in place of 6, so FP6^{g} costs $6 + 8/16 = 6.5$ bits per element. Table 6 lists the positive magnitudes of both element formats.

B.3 Two-Bit Formats and the AutoFP2 Digit

With one bit for the sign, a 2-bit floating-point format spends its other bit on either the exponent or the mantissa. **E1M0** spends it on the exponent: giving the magnitudes $\{1, 2\}$ and the codebook $\{\pm 1, \pm 2\}$, which cannot represent zero, so small values are forced to ± 1. **E0M1** spends it on the mantissa: both codes are subnormal, giving the ternary codebook $\{-1, 0, +1\}$ with a redundant -0, which represents zero but only one nonzero magnitude. AutoFP2's quinary digit $\{-2, -1, 0, 1, 2\}$ keeps both by remapping the two exponent bits of the NVFP4 base (§3.2).

Table 7. Mantissa refinement of one NVFP4 group ($s = 0.25$). Values are in NVFP4 units; codes are sign — exponent — mantissa. Green rows are the refined subgroups (mask `0110`).

Subgroup	x/s	NVFP4 base			AutoFP6 refinement			
		q	Code	Err.	δ	Code	$q+\delta$	Err.
SG0	1.00	1.0	0 0 1 0	0	–	–	–	–
	−0.50	−0.5	1 0 0 1	0	–	–	–	–
	3.00	3.0	0 1 0 1	0	–	–	–	–
	6.00	6.0	0 1 1 1	0	–	–	–	–
SG1	1.13	1.0	0 0 1 0	0.13	$+\frac{1}{8}$	01	**1.125**	**0.005**
	2.24	2.0	0 1 0 0	0.24	$+\frac{1}{8}$	01	**2.125**	**0.115**
	−0.87	−1.0	1 0 1 0	0.13	$+\frac{1}{8}$	01	**−0.875**	**0.005**
	1.37	1.5	0 0 1 1	0.13	$-\frac{1}{8}$	11	**1.375**	**0.005**
SG2	2.76	3.0	0 1 0 1	0.24	$-\frac{1}{4}$	10	**2.75**	**0.010**
	−0.62	−0.5	1 0 0 1	0.12	$-\frac{1}{8}$	11	**−0.625**	**0.005**
	4.24	4.0	0 1 1 0	0.24	$+\frac{1}{8}$	01	**4.125**	**0.115**
	−2.02	−2.0	1 1 0 0	0.02	0	00	**−2.0**	**0.020**
SG3	0.52	0.5	0 0 0 1	0.02	–	–	–	–
	−1.98	−2.0	1 1 0 0	0.02	–	–	–	–
	0.00	0.0	0 0 0 0	0	–	–	–	–
	2.95	3.0	0 1 0 1	0.05	–	–	–	–

B.4 Sharing NVFP4 Scales with AutoFP2

A tensor stored in NVFP4 that executes at AutoFP2 reuses the NVFP4 scales T and B_j. These normalize each group to $[-6, 6]$, whereas AutoFP2 digits reach only ± 2, so decoding with TB_j alone would shrink each group by up to $3\times$. The scale is therefore multiplied by a scale multiple c:

$$\widehat{x}_i = cTB_j\, q_i, \qquad q_i \in \{-2, -1, 0, 1, 2\}. \tag{4}$$

Setting $c = 3$ matches the two ranges exactly, with reconstruction levels $\{0, \pm 3, \pm 6\}$ in NVFP4 units. We instead use $c = 2$, with levels $\{0, \pm 2, \pm 4\}$, for two reasons. First, a factor of two is hardware-friendly. Second, the AutoFP2 digit 1 absorbs the four E2M1 magnitudes $\{1, 1.5, 2, 3\}$, which hold most group-normalized values. Reconstructing this digit at 2 rather than 3 places it in the middle of that range and halves its worst-case error from 2 to 1 NVFP4 units.

B.5 Worked Example of Mantissa Refinement

Table 7 applies mantissa refinement to one 16-element NVFP4 group with scale $s = TB_j = 0.25$.

C Additional Motivating Analyses

C.1 Layer-Wise Precision Across Decode Steps

Figure 16 shows the oracle per-layer precision at individual decode steps.

C.2 Layer-Wise Precision Across Prefill and Refills

Figure 17 repeats the prefill and refill analysis of Figure 2(b) for an additional model and workload, Gemma-4-26B on LongMemEval, showing the oracle layer-wise precision of the initial prefill and three refills for three samples.

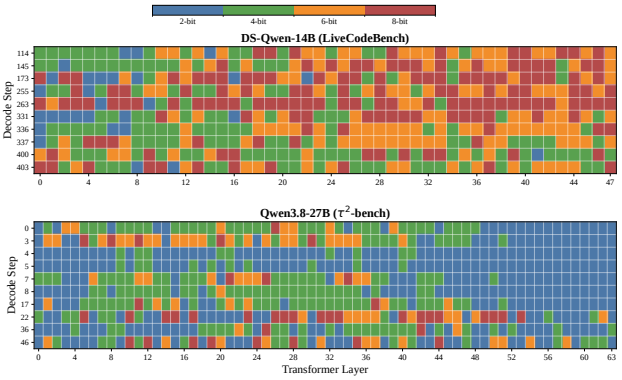


Figure 16. **Layer-wise precision across decode steps.** Per-layer precision at ten decode steps of one DS-Qwen-14B LiveCodeBench response (top) and one Qwen3.8-27B τ^2-bench response (bottom).

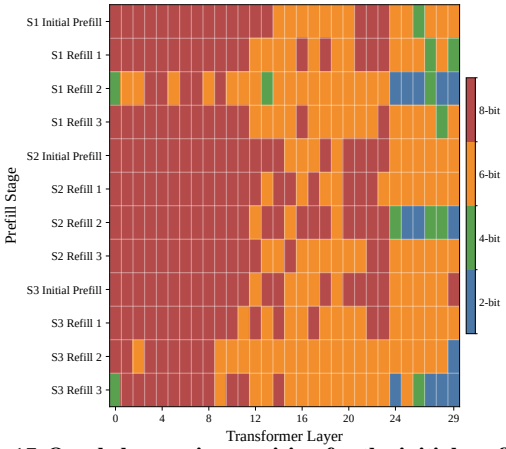


Figure 17. **Oracle layer-wise precision for the initial prefill and three refills** of Gemma-4-26B on three LongMemEval samples (S1–S3).

C.3 Precision Demand by Generation Entropy

Figure 18 splits the decode steps of the oracle of §2 by $\mathcal{G}_\mathcal{E}$ at $\tau = 0.10$. High-entropy steps consistently require higher precision compared to low-entropy steps.

C.4 Layer-Wise Behavior of $\mathcal{M}_P$ and $\mathcal{M}_D$ Across Models and Datasets

Figure 30 extends Figure 4 to Gemma-4-26B, Qwen3.8-27B, Qwen3-8B, and DS-LLaMA-70B on LiveCodeBench, GSM8K, and AIME. Figure 19 shows $\mathcal{M}_P$ of the initial prefill and three refills of one τ^2-bench task for Gemma-4-26B and Qwen3.8-27B.

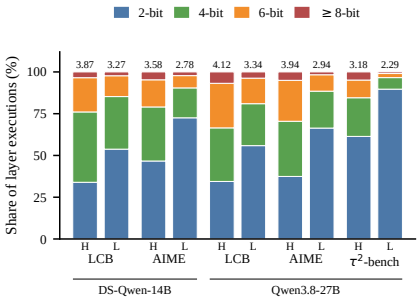


Figure 18. **Oracle precision demand in high- (H) and low-entropy (L) decode steps.**

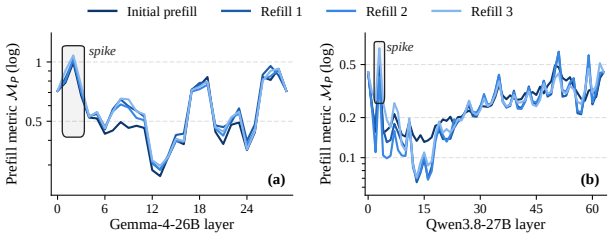


Figure 19. **Prefill $\mathcal{M}_P$ for the initial prefill and three refills** of one τ^2-bench task for (a) Gemma-4-26B and (b) Qwen3.8-27B.

C.5 Subgroup Refinement on Additional Models

Figure 20 repeats the experiment of Figure 3 on Qwen3.8-27B and Gemma-4-26B with GSM8K, for three layers each.

D Additional AutoEMatic Details

D.1 Signals Used for Runtime Precision Adaptation

Figure 21 illustrates the two model-internal signals of §3.1.

D.2 Entropy Signal Across Decode Steps

Figure 22 traces $\mathcal{G}_\mathcal{E}$, over a 16,383-step AIME generation.

D.3 Memory Layout

Figure 23 shows the memory layout of AutoFP, with the NVFP4 base stored separately from the refinement metadata.

D.4 Extended-K GEMM

Figure 24 illustrates the identity behind the extended-K GEMM of §4.1 on a numeric example: two GEMMs with the same output shape sum to one GEMM over operands concatenated along K.

D.5 Construction of the Extended-K Operands

Figure 25 shows how $\mathbf{W}_{qr}$ and $\mathbf{A}_{qr}$ are constructed.

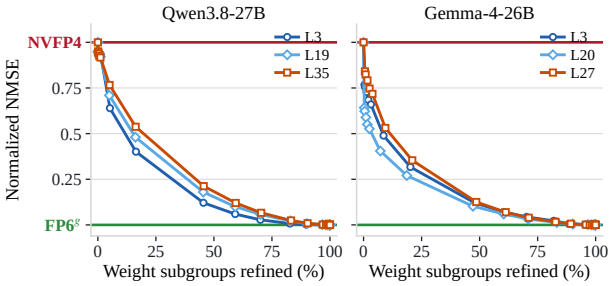


Figure 20. **Refining a subset of elements achieves NMSE comparable to 6-bit**. Y-axis is NMSE, such that NVFP4 is 1 and FP6^g is 0, vs. fraction of subgroups refined to 6-bit, for Qwen3.8-27B and Gemma-4-26B on GSM8K.

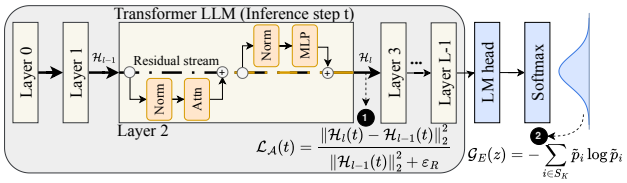


Figure 21. **Signals used by AutoEMatic for runtime precision adaptation.** ❶ Local layer-update activity $\mathcal{L}_\mathcal{A}$. ❷ Global token-generation uncertainty $\mathcal{G}_\mathcal{E}$.

D.6 Numerical Example of the Extended-K GEMM

Figure 26 shows a worked example of the GEMM $[\mathbf{W}_q \,|\, \mathbf{W}_{qr}]\,[\mathbf{A}_q \,|\, \mathbf{A}_{qr}]$:

$$\begin{aligned}\widehat{\mathbf{W}}\widehat{\mathbf{A}}^\top &= \mathbf{W}_q\mathbf{A}_q^\top + \tfrac{s_W}{4}\,\mathbf{W}_{qr}\mathbf{A}_{qr}^\top \\ &= [\mathbf{W}_q \,|\, \mathbf{W}_{qr}]\,[\mathbf{A}_q \,|\, \mathbf{A}_{qr}]^\top \\ &= 9.25 + \tfrac{1}{4}\big(0.5\cdot 2 + 3\cdot(-0.5)\big) = 9.125.\end{aligned} \tag{5}$$

D.7 NVFP4-to-AutoFP2 Decomposition

The weight encoder decomposes each NVFP4 weight into two AutoFP2 digits, $W_q = c_0 Q_0 + c_1 Q_1$ with $Q_0, Q_1 \in \{-2, \ldots, 2\}$ (§6). With $c_1 = 1$, to span the maximum NVFP4 magnitude 6 requires $2c_0 + 2 \geq 6$. $c_0 = 2.5$ satisfies this such that $c_0 Q_0$ provides coarse-grained steps to different NVFP4 binades and $c_1 Q_1$ provides integer corrections $[-2, 2]$, such that their linear combination spans every NVFP4 E2M1 value. Multiplying numerator and denominator by 2 gives integer coefficients such that, $W_q = (5Q_0 + 2Q_1)/2$ and the resulting reconstruction can be realized using only shifts and adds. Table 8 lists the digit pair of every positive E2M1 magnitude; negative weights negate both digits.

For each digit Q_i, the encoder produces three bits from the E2M1 bits $\{s, e_1, e_0, m\}$ of the weight: a valid bit v_i, which is 1 if $Q_i \neq 0$ and 0 if $Q_i = 0$; a sign bit s_i; and an exponent bit $e_{q,i}$ with $|Q_i| = 2^{e_{q,i}}$, so that $\{s_i, e_{q,i}\}$ is the

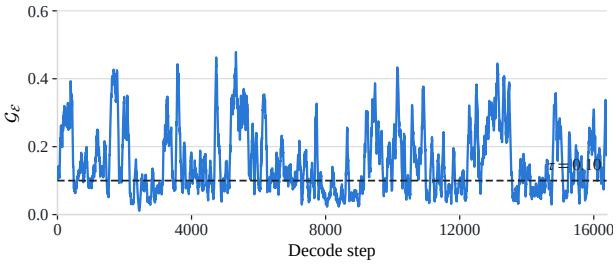


Figure 22. **Global signal $\mathcal{G}_\mathcal{E}$ over a 16K-token AIME generation of Qwen3.8-27B.** The dashed line marks $\tau = 0.10$.

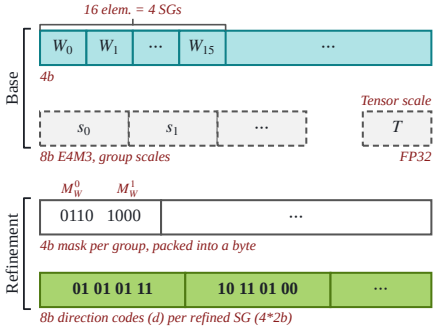


Figure 23. **AutoEMatic memory layout.**

E1M0 code of a nonzero digit:

$$\begin{aligned} v_0 &= m \vee (e_1 \wedge e_0), & v_1 &= e_1 \vee e_0 \vee m, \\ e_{q,0} &= e_1 \wedge (e_0 \vee m), & e_{q,1} &= \neg e_0 \wedge (e_1 \vee m), \\ s_0 &= s, & s_1 &= s \oplus m \oplus (e_1 \wedge e_0), \end{aligned} \tag{6}$$

where $\vee$, $\wedge$, $\neg$, and $\oplus$ denote OR, AND, NOT, and XOR. The valid bits then select the AutoFP2 encoding of §3.2: if both digits are nonzero ($v_0 = v_1 = 1$), the escape bit is $E = 0$ and the fields F_1 and F_0 hold the two E1M0 codes; otherwise $E = 1$, F_1 records which digits are zero, and F_0 holds the E1M0 code of the nonzero digit, if any. The valid bits are consumed by this packing and are not stored.

D.8 Fixed-Point Arithmetic of the vMAC

All computation in the vMAC (§6.3) is fixed-point. Each input activation enters the lanes as the integer $A_k = 2A_q(k)$, since doubling an E2M1 value always yields an integer (Hayashi et al., 2026). AutoFP2 digits are already integers, whereas in NVFP4 mode the packed digit pair represents twice the NVFP4 weight, $2W_q = 5Q_0 + 2Q_1$ (§6.2). In the multiply stage each sub-lane forms $A_k F_{j,k}$, $j \in \{0, 1\}$, and within each block the even and odd sub-lanes feed separate adder trees, $A = \sum_{k=0}^{15} A_k F_{0,k}$ and $B = \sum_{k=0}^{15} A_k F_{1,k}$ for lanes 0–15, and C, D for lanes 16–31.

In NVFP4 mode, A and B are partial sums over the two digits of the same weights $W_{q,k}$, so by distributivity the

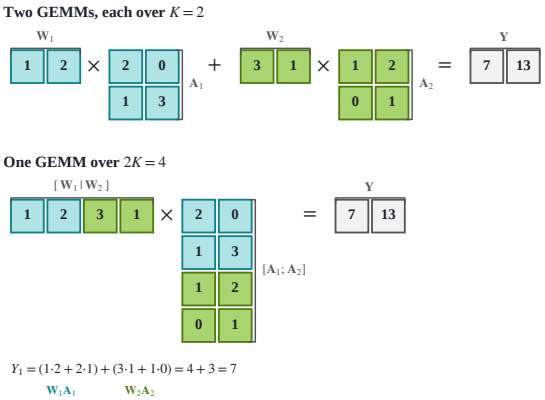


Figure 24. **A sum of GEMMs as one GEMM over an extended reduction dimension.**

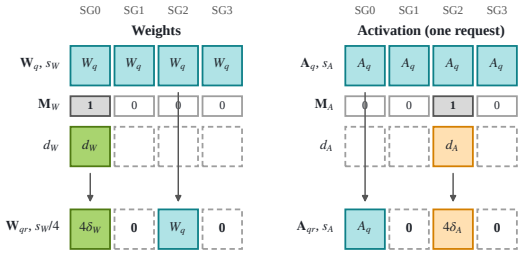


Figure 25. **Construction of $\mathbf{W}_{qr}$ and $\mathbf{A}_{qr}$ for one 16-element group and one request.** The weight mask places the weight offsets $4\delta_W$ in $\mathbf{W}_{qr}$ and copies the base A_q into $\mathbf{A}_{qr}$; the activation mask copies the base W_q into $\mathbf{W}_{qr}$ and places the activation offsets $4\delta_A$ in $\mathbf{A}_{qr}$; all other positions are zero.

mode-dependent reduction unit evaluates $P_0 = 5A + 2B = \sum_k A_k(5F_{0,k} + 2F_{1,k}) = \sum_k (2A_q)(2W_q)$, which is four times the dot product of block 0 in fixed-point; the division by two in $(5A + 2B)/2$ of §6.3 is deferred to the scaling stage. Similarly for P_1. In AutoFP2 mode, the even and odd sub-lanes of a block hold weights from two different rows, so the partial sums are summed across blocks as $P_0 = 2(A+C)$ and $P_1 = 2(B+D)$, two independent 32-element dot products; the factor of 2 keeps them at four times the dot product as well.

Each refinement lane forms $R = 2X_q d$ in fixed-point. Since $\delta = d \cdot 2^{-3}$, the floating-point equivalent is $X_q\delta = R/16$, so R is sixteen times the product. The lane contributions are accumulated as $\mathcal{R} = \sum R$, and the base sums P_j are shifted left by two bits to align with them, giving $O_j = 4P_j + \mathcal{R}$ at sixteen times the dot product. For example, with $A_q = 1.5$, $W_q = 3$, and $d_W = -1$: $P = (2\cdot1.5)(2\cdot3) = 18$, $R = -3$, and $O = 69 = 16 \cdot 1.5\,(3 - 1/8)$. In the scaling stage, each O_j is multiplied by the scale factor of its block and divided by 16, which recovers the floating-point partial sum.

D.9 Accelerator Floorplan

Figure 27 shows the AutoEMatic accelerator floorplan.

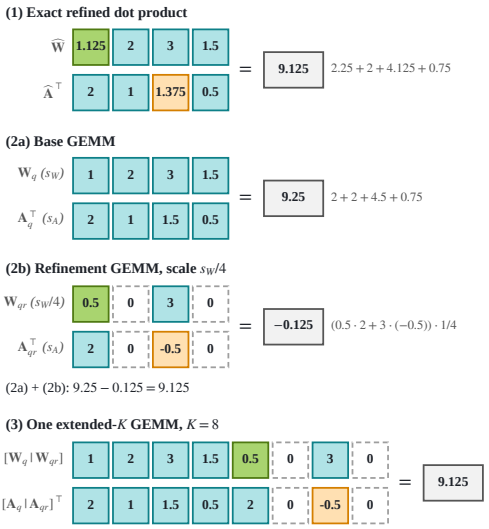


Figure 26. (1) The exact product of the refined operands $\widehat{\mathbf{W}}$ and $\widehat{\mathbf{A}}$. (2) The base GEMM on $\mathbf{W}_q$ and $\mathbf{A}_q$ plus a refinement GEMM on the constructed $\mathbf{W}_{qr}$ and $\mathbf{A}_{qr}$ at scale $s_W/4$. (3) A single GEMM over the concatenated operands.

Table 8. NVFP4-to-AutoFP2 decomposition.

W_q	0	0.5	1	1.5	2	3	4	6
Q_0	0	1	0	1	0	2	2	2
Q_1	0	−2	1	−1	2	−2	−1	1

E Refinement-Aware GPTQ Calibration

Algorithm 1 lists AutoEMatic's offline weight calibration, which extends GPTQ (Frantar et al., 2023) to an NVFP4 base with N:4 structured sparse refinement. We follow the NVFP4 notation of §B.1: T is the FP32 tensor scale, and $B_{n,j}$ is the E4M3 group scale of row n in group j, whose 16 columns we denote J. The calibration stores E2M1 codes $q_{n,i}$ and, on refined subgroups, 2-bit signed offsets $d_{n,i}$, decoded with the fixed step of §3.2 as $\delta_{n,i} = d_{n,i}\,2^{-3} \in \mathcal{D} = \{-\frac{1}{4}, -\frac{1}{8}, 0, +\frac{1}{8}\}$; the dequantized weight is $TB_{n,j}\,q_{n,i}$ without refinement (NVFP4) and $TB_{n,j}(q_{n,i}+\delta_{n,i})$ with it (AutoFP6). A refinement is set to 0 if it would move the element farther from the original weight.

Each group has four 4-element subgroups, and $\mathcal{M}_\ell$ denotes the masks that refine ℓ of them; the weight mask $\mathbf{M}_W$ of §3.2 holds one such mask m per group. For a mask m, $\widehat{\mathbf{W}}(m)$ is the dequantized group that uses the refined values $\widehat{\mathbf{W}}^r$ on the selected subgroups and the NVFP4 values $\widehat{\mathbf{W}}^4$ elsewhere. Its diagonal-Hessian-weighted error on group j is

$$\mathcal{E}_j(m) = \sum_{i \in J} \mathbf{H}_{ii} \left\| \mathbf{W}_{:,i} - \widehat{\mathbf{W}}_{:,i}(m) \right\|_2^2. \tag{7}$$

The algorithm differs from GPTQ in three ways. **(1) Budget allocation.** Before the GPTQ pass, a greedy allocation (lines 5–12) distributes $\rho K/4$ refined subgroups across groups, ranked by their reduction in $\mathcal{E}_j$ on the original weights. Each group's mask, which refines N_W of its four subgroups, is shared across all output rows; the remaining $N_A = 4 - N_W$ subgroups are left for activation refinement at runtime. **(2) Group-wise quantization.** The 16-element group, rather than a single column, is the unit of quantization. Its group scales $B_{n,j}$ are computed from the error-compensated weights when the group is reached, while the tensor scale T stays fixed. **(3) Group-wise error propagation.** The quantization error of a whole group is propagated to the later columns at once (lines 22–23), rather than one column at a time as in GPTQ.

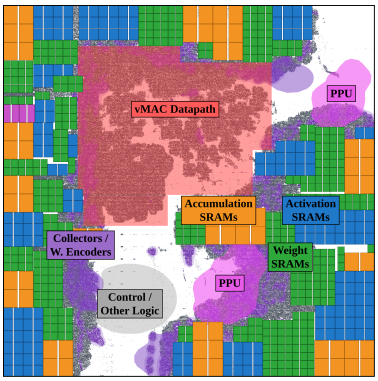


Figure 27. **Post-layout floorplan of AutoEMatic**.

F GPU Implementation Details

AutoEMatic is implemented in vLLM 0.15 (Kwon et al., 2023) as a quantization plugin, with a custom attention backend and CUDA kernels for Blackwell GPUs.

G Accelerator Simulation Details

AutoEMatic performance is measured using a scheduling model calibrated against cycle-accurate Synopsys VCS simulation. Performance data is gathered from these measurements, retaining on-chip data-movement and backpressure costs. Off-chip DRAM transfer time is modeled as transferred bytes divided by an assumed sustained bandwidth of 1 TB/s and added to the execution time.

H Additional Accuracy Results

Table 9 reports the accuracy of AutoEMatic and the baselines on DS-Llama-70B, DS-Qwen-32B, and Llama-3.1-8B-Instruct on the reasoning benchmarks.

I Additional Ablations

AutoFP2 scale multiple c. Figure 28(a) sweeps the scale multiple c of §B.4: $c = 2$ gives an NMSE of 0.09, within

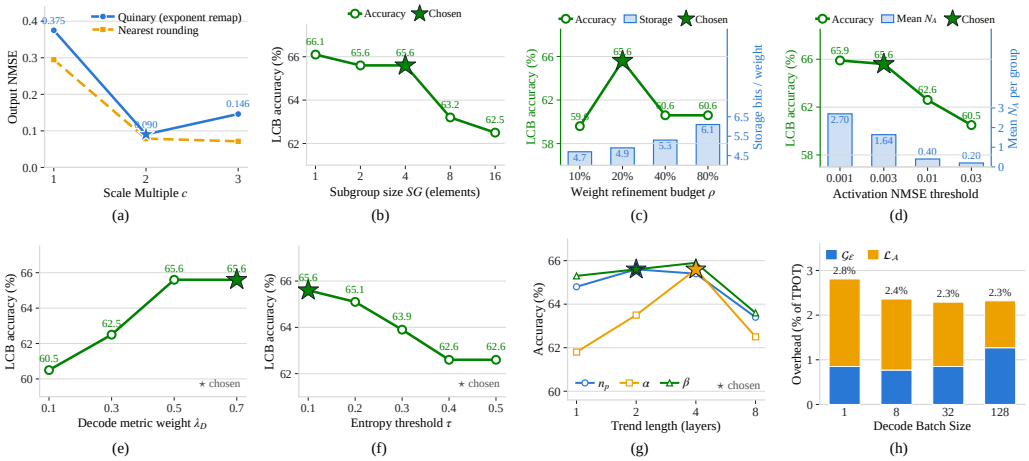


Figure 28. **Additional ablations** on DS-LLaMA-70B (LCB). (a) Scale multiple c of AutoFP2: mean layer-output NMSE over all 80 layers at W2A4. (b) Subgroup size SG with 20% of all weight subgroups refined. (c) Weight refinement budget ρ, with storage precision. (d) Activation NMSE threshold, with the mean number of activation subgroups per group exceeding it. (e) λ_D sweep. (f) Entropy threshold τ. (g) Trend lengths n_p, α, and β. (h) Overhead of computing $\mathcal{G}_\mathcal{E}$ and $\mathcal{L}_\mathcal{A}$ as a share of decode TPOT. Stars mark the chosen settings.

Table 9. **Accuracy of AutoEMatic and baselines on DS-Llama-70B, DS-Qwen-32B, and Llama-3.1-8B-Instruct.** Cells report accuracy (%); execution precision appears in parentheses. LB v2: LongBench v2, LCB: LiveCodeBench. Llama-3.1-8B-Instruct is not evaluated on AIME and LCB, where its BF16 accuracy is too low for a meaningful comparison.

Model	Method (budget)	LB v2	AIME	LCB	GSM8K
DS-Llama-70B	BF16	41.0	66.7	68.2	94.5
	NVFP4	29.8	53.3	51.7	90.6
	FP6^g	**41.0**	**66.7**	**66.2**	**94.5**
	AutoQuantize (5.3)	38.7	**66.7**	55.7	**94.5**
	FlexQuant (5.3)	27.8	50.0	48.6	86.5
	DP-LLM (weight-only) (5.3)	21.0 (5.1)	43.3 (5.3)	43.3 (5.3)	81.6 (5.1)
	DP-LLM† (5.3)	19.0 (5.1)	36.7 (5.3)	39.5 (5.3)	77.6 (5.1)
	AutoEMatic	**41.0** (5.4)	**66.7** (5.1)	65.6 (4.5)	**94.5** (4.9)
DS-Qwen-32B	BF16	49.8	73.3	54.2	92.5
	NVFP4	43.5	60.0	50.5	87.6
	FP6^g	**49.8**	**70.0**	**52.6**	**90.5**
	AutoQuantize (4.8)	**49.8**	66.7	51.2	89.3
	FlexQuant (4.8)	40.5	56.7	46.9	81.5
	DP-LLM (weight-only) (4.8)	36.6 (4.8)	50.0 (4.8)	45.0 (4.8)	76.5 (4.5)
	DP-LLM† (4.8)	30.6 (4.8)	36.7 (4.8)	40.7 (4.8)	71.8 (4.5)
	AutoEMatic	**49.8** (4.6)	**70.0** (4.8)	52.4 (4.6)	**90.5** (4.4)
Llama-3.1-8B-Instruct	BF16	30.5	–	–	82.8
	NVFP4	27.5	–	–	75.6
	FP6^g	**30.5**	–	–	**80.5**
	AutoQuantize (4.5)	**30.5**	–	–	78.9
	FlexQuant (4.5)	25.9	–	–	71.6
	DP-LLM (weight-only) (4.5)	24.6 (4.4)	–	–	63.8 (4.3)
	DP-LLM† (4.5)	22.8 (4.4)	–	–	59.6 (4.3)
	AutoEMatic	**30.5** (4.4)	–	–	**80.5** (4.1)

$1.15\times$ of nearest rounding, whereas $c = 1$ clips the large magnitudes and $c = 3$ places the merged E2M1 codes $\{1, 1.5, 2, 3\}$ at 3, raising the NMSE to 0.38 and 0.15.

Impact of second-order refinement. Figure 31 measures the second-order refinement product $\mathbf{W}_r\mathbf{A}_r$ that the extended-K GEMM omits (§4.1). Its energy is 1.4×10^{-6} of the per-layer output, and consequently the AutoFP6 output NMSE is 0.0041 with and without this term.

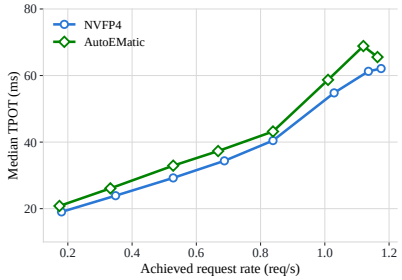


Figure 29. **Online serving under continuous batching.** Median TPOT vs. achieved request rate for DS-LLaMA-70B on $4\times$ GB300 (TP=4) with Poisson arrivals of LongBench v2 requests.

Table 10. **Extension to 8 bits** with DS-LLaMA-70B on LCB; rows 1 and 7 of Table 4 are repeated for reference.

Component	Bits	Acc.	2 / 4 / 6 / 8 (%)
BF16	16	68.4	–
AutoEMatic format + runtime policy, $\mathcal{P} = \{2, 4, 6\}$	4.9	65.6	15 / 45 / 40 / 0
AutoFP8 + Refinement-aware GPTQ	5.3	67.9	0 / 0 / 0 / 100
AutoEMatic format + runtime policy, $\mathcal{P} = \{2, 4, 6, 8\}$	5.3	**67.9**	30 / 35 / 15 / 20

Extension of K. The reduction extension of a refined GEMM depends on the # of subgroups refined. On Qwen3.8-27B τ^2-bench (Figure 32), the extension grows from $0.49K$ at batch 1 to $0.81K$ at batch 32 with average refined requests per layer rising from 0.2 to 7.2.

Extension to 8 bits. Widening the signed offsets from 2 to 4 bits extends AutoEMatic to AutoFP8 (Table 10). With every layer statically refined to AutoFP8 and refinement-aware GPTQ, DS-LLaMA-70B attains 67.9% on LCB at 5.3 stored bits per weight. The runtime policy with $\mathcal{P} = \{2, 4, 6, 8\}$ attains the same 67.9%, within 0.5% of BF16, at 5.3 bits while executing 30% of layers at 2-bit, 35% at 4-bit, 15% at 6-bit, and 20% at 8-bit.

Subgroup size. Figure 28(b) varies SG with 20% of all

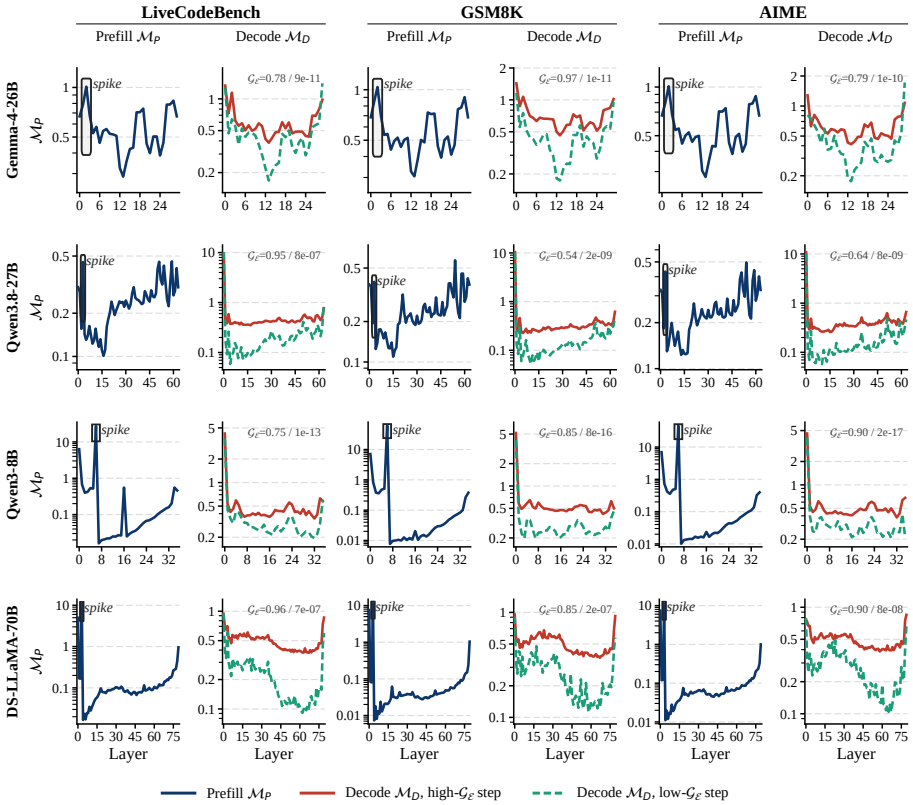


Figure 30. **Layer-wise $\mathcal{M}_P$ and $\mathcal{M}_D$ across models and datasets.** For each model (rows) and dataset (column pairs), prefill $\mathcal{M}_P$ of one representative sample and decode $\mathcal{M}_D$ of the same sample at the highest- and lowest-$\mathcal{G}_\mathcal{E}$ step.

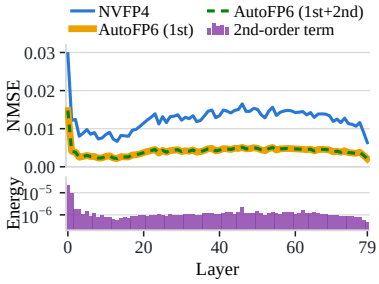


Figure 31. **Second-order refinement term.** Per-layer output NMSE of AutoFP6 with and without the second-order product $\mathbf{W}_r\mathbf{A}_r$ (top) and the energy of that product relative to the layer output (bottom) on DS-LLaMA-70B.

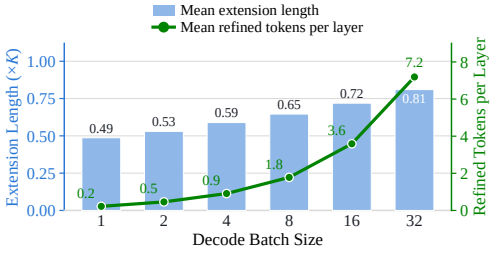


Figure 32. **Extension of the reduction dimension** of refined GEMMs on Qwen3.8-27B τ^2-bench vs. decode batch size, with the mean number of refined tokens per layer.

weight subgroups refined. Finer subgroups raise accuracy only from 65.6% at $SG = 4$ to 66.1% at $SG = 1$, however, finer subgroups require more metadata per group and are not as performant. Coarser subgroups lose 2.4–3.1%, as a refined subgroup then spends signed offsets on elements that do not need them.

Weight refinement budget. Figure 28(c) sweeps ρ, the fraction of all weight subgroups refined, over $\{10, 20, 40, 80\}$%, with storage of $4.5 + 2\rho$ bits per weight. Accuracy peaks at $\rho = 20\%$ (65.6% at 4.9 bits) and falls on both sides: at 10% too few weight subgroups are refined, while at 40% and 80% the weight mask occupies subgroups that the activations would otherwise refine at runtime ($N_A = 4 - N_W$), so additional weight bits displace activation refinement and accuracy drops to 60.6%.

Activation refinement threshold. Figure 28(d) sweeps the NMSE threshold above which an activation subgroup is refined, over $\{0.001, 0.003, 0.01, 0.03\}$, together with the mean number of subgroups per group that exceed it. Lowering the threshold below 0.003 refines 2.7 rather than 1.6 subgroups per group for 0.3% more accuracy, while raising it to 0.01 and 0.03 leaves most groups unrefined.

λ_D. Figure 28(e) sweeps λ_D over $\{0.1, 0.3, 0.5, 0.7\}$.

Algorithm 1 Refinement-aware GPTQ calibration of $\mathbf{W} \in \mathbb{R}^{N\times K}$, given calibration inputs $\mathbf{X} \in \mathbb{R}^{M\times K}$, group size $G = 16$, refinement budget ρ, and damping ω. Returns the E2M1 codes $q_{n,i}$, the refinement terms $\delta_{n,i}$ (stored as 2-bit signed offsets) on refined subgroups, the scales T and $B_{n,j}$, and the masks $m_j^\star$.

$\mathbf{H} \leftarrow \mathbf{X}^\top\mathbf{X}/M$
$\mathbf{H}_\omega \leftarrow \mathbf{H} + \omega\,\mathrm{mean}(\mathrm{diag}\,\mathbf{H})\,\mathbf{I}$
$\mathbf{U} \leftarrow \mathrm{Cholesky}(\mathbf{H}_\omega^{-1})^\top$
$T \leftarrow \mathrm{amax}(\mathbf{W})/(6\cdot 448)$ ▷ fixed
▷ Allocate refinement budget on the original weights
for $j = 0$ **to** $K/G - 1$ **do**
  $L_j(\ell) \leftarrow \min_{m\in\mathcal{M}_\ell} \mathcal{E}_j(m)$ for $\ell \in \{0,1,2\}$
end for
$\ell_j \leftarrow 0$ for all j
for $b = 1$ **to** $\rho K/4$ **do**
  $j^\star \leftarrow \arg\max_{j:\ell_j<2}\ L_j(\ell_j) - L_j(\ell_j+1)$
  $\ell_{j^\star} \leftarrow \ell_{j^\star} + 1$
end for
▷ GPTQ pass over groups
for $j = 0$ **to** $K/G - 1$ **do**
  $J \leftarrow jG : (j+1)G$
  $B_{n,j} \leftarrow Q_{\mathrm{E4M3}}\big(\mathrm{amax}(\mathbf{W}_{n,J})/6T\big)$ for all n
  $q_{n,i} \leftarrow Q_{\mathrm{E2M1}}\big(\mathbf{W}_{n,i}/TB_{n,j}\big)$ for all $n, i \in J$
  $\delta_{n,i} \leftarrow \arg\min_{\delta\in\mathcal{D}} \big|\mathbf{W}_{n,i}/TB_{n,j} - q_{n,i} - \delta\big|$
  $\widehat{\mathbf{W}}^4_{n,i} \leftarrow TB_{n,j}\,q_{n,i}$ ▷ dequantized
  $\widehat{\mathbf{W}}^r_{n,i} \leftarrow TB_{n,j}\,(q_{n,i} + \delta_{n,i})$ ▷ dequantized
  $m_j^\star \leftarrow \arg\min_{m\in\mathcal{M}_{\ell_j}} \mathcal{E}_j(m)$
  $\widehat{\mathbf{W}}_{:,J} \leftarrow \widehat{\mathbf{W}}(m_j^\star)$
  $\mathbf{E} \leftarrow (\mathbf{W}_{:,J} - \widehat{\mathbf{W}}_{:,J})\,\mathbf{U}_{J,J}^{-1}$
  $\mathbf{W}_{:,(j+1)G:} \leftarrow \mathbf{W}_{:,(j+1)G:} - \mathbf{E}\,\mathbf{U}_{J,(j+1)G:}$
end for

Entropy threshold. Figure 28(f) sweeps τ, which separates the high- and low-entropy decode regimes, over $\{0.1,\ldots,0.5\}$. Accuracy is highest at $\tau = 0.1$ (65.6%) and declines monotonically to 62.6% at $\tau \geq 0.4$, as more steps with several plausible continuations are treated as low-entropy and allowed to lower precision aggressively.

Trend lengths. Figure 28(g) sweeps n_p, α, and β over $\{1,2,4,8\}$.

Signal overhead. Figure 28(h) reports the cost of computing the signals as a share of decode TPOT. $\mathcal{G}_\mathcal{E}$ is one top-K selection and entropy over the logits per step, below 1.3% at every batch size; $\mathcal{L}_\mathcal{A}$ is a per-layer reduction of the residual update, 2.4 μs per layer, whose share falls from 2.0% at batch 1 to 1.1% at batch 128 as it amortizes over rows. Together the signals add 2.3–2.8% of TPOT.

Online serving. Figure 29 serves DS-LLaMA-70B in vLLM under continuous batching with Poisson request arrivals at increasing rates, using LongBench v2 prompts of about 8K tokens with 1–2K output tokens.

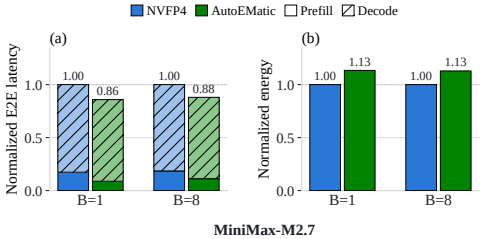


Figure 33. **Accelerator latency and energy at iso-accuracy with NVFP4.**

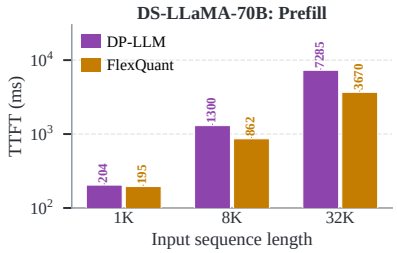


Figure 34. **TTFT of the runtime-adaptive baselines** DP-LLM and FlexQuant on DS-LLaMA-70B vs. input sequence length.

Table 11. **Instantaneous vs. trend-based precision changes** with Qwen3-8B on GSM8K.

Policy	Bits	Acc.	2 / 4 / 6 (%)
Instantaneous (n_p=α=β=1)	4.7	92.5	0 / 66 / 34
AutoEMatic (n_p=2, α=4, β=2)	4.6	**95.5**	0 / 69 / 31

Accelerator latency and energy at iso-accuracy with NVFP4. The main comparison (§6) holds accuracy at AutoEMatic's level. Figure 33 instead holds it at NVFP4's. This enables the NVFP4 datapath to run all layers at its native precision. To achieve iso-accuracy with NVFP4, we tune AutoEMatic assignments on MiniMax-M2.7 such that AutoEMatic assigns 30% of layers to AutoFP2, 60% to NVFP4, and 10% to AutoFP6 and matches the accuracy of the NVFP4. As shown in Figure 33, AutoEMatic still reduces end-to-end latency by 12–14%, with prefill 1.7–2.0× faster at batch sizes 1 and 8, at 13% higher energy, due to the additional components in the mixed-precision AutoEMatic datapath.

DP-LLM and FlexQuant performance. Figure 34 reports the prefill latency of DP-LLM and FlexQuant on DS-LLaMA-70B. At batch size 1 they decode at 5.9 and 7.3 tokens/s, respectively, against 131 tokens/s for NVFP4 and 120 for AutoEMatic in Figure 8.

Instantaneous vs. trend-based changes. Table 11 compares making precision decisions instantaneously versus AutoEMatic's trend-based policy. The trend-based policy reaches both lower average execution precision (4.6 vs. 4.7 bits) and higher accuracy (95.5% vs. 92.5%). Instantaneous decisions react to layer-to-layer noise in the metric, the trend

rule acts only on sustained changes, which filters this noise.

J LIMITATIONS

As it stands, while AutoFP supports the precision set $\mathcal{P} = \{2, 4, 6\}$, it cannot take advantage of AutoFP2's memory bandwidth benefits. Using AutoFP2 as the stored base would require reconstructing NVFP4 for most layer executions, introducing overhead that can outweigh the bandwidth savings during decode. Furthermore, AutoFP2's coarse representation requires larger corrections to recover NVFP4 values, which the current signed-offset encoding does not directly support. Future work will work on remedying this.

K AI USAGE STATEMENT

Portions of this paper were refined with the assistance of a large language model (LLM), specifically OpenAI GPT 6-Sol, used to polish writing and help reduce verbosity to meet page limit. The LLM also assisted in writing the python code for making several figures for the data the authors independently collected. All technical content, methods, and results were conceived entirely by the authors, without influence from any AI tool. The AI was however used to run parameter sweeps, conduct design space exploration, suggest optimizations (which the authors thoroughly verified) and also launch and monitor accuracy and performance experiments.